MISTv: Multimedia, Internet, Storage, and Television

Undergraduate Project

by

Wilbert Jethro Ramirez Limjoco
2009-52731
B.S. Electronics and Communications Engineering

Philip Bernard Francisco Nery
2009-54042
B.S. Computer Engineering

Shulamith Rivera Sevilla
2007-46513
B.S. Electronics and Communications Engineering

Adviser:

Dr. Roel M. Ocampo

University of the Philippines, Diliman
April 2014

Certification

We certify on our honor that this document had been duly approved by our project adviser and all information herewith are true and correct to the best of our knowledge. I understand that any falsification of information will be subject to the appropriate penalties, which may include an automatic failure of EE/CoE/ECE 198.

Approval Sheet

In partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Engineering and Electronics and Communications Engineering, this project proposal entitled "MISTv: Multimedia, Internet, Storage, and Television", prepared by Wilbert Jethro R. Limjoco, Philip Bernard F. Nery, and Shulamith R. Sevilla is hereby recommended for approval.

Roel M. Ocampo, Ph.D.
Adviser

Edwin M. Umali, Ph.D.
Examiner

UNIVERSITY OF THE PHILIPPINES

Bachelor of Science in Electronics and Communications Engineering

Wilbert Jethro R. Limjoco

Bachelor of Science in Computer Engineering

Philip Bernard F. Nery

Bachelor of Science in Electronics and Communications Engineering

Shulamith R. Sevilla

MISTv: Multimedia, Internet, Storage, and Television

Undergraduate Project Adviser:

Roel M. Ocampo, PhD

Electrical and Electronics Engineering Institute

University of the Philippines Diliman

Undergraduate Project Reader:

Edwin M. Umali, PhD

Electrical and Electronics Engineering Institute

University of the Philippines Diliman

Date of Submission

04 April 2014

Permission is given for the following people to have access to this thesis:

Circle one or more concerns: I P C	
Available to the general public	Yes/No
Available only after consultation with author/thesis adviser	Yes/No
Available only to those bound by confidentiality agreement	Yes/No

Students' signature/s:

Signature/s of undergraduate project advisers:

Acknowledgements

I would like to thank the people who have helped me get this far.

- The complex higher powers for letting me exist in the universe.
- Our adviser, Dr. Roel M. Ocampo, for giving us the opportunity to work on this project under his excellent guidance.
- My groupmates, Philip Nery and Mithi Sevilla, for deciding to tag along with me for this project.
- My parents, for raising me well and funding my education.
- My best friends, for being there with me in the fun times and in the sad times.
- Our lab head, Dr. Rowel O. Atienza, for accepting us in the Computer Networks Laboratory and for letting me become an SA for EEE 11 and EEE 13
- Kuya Lope Beltran, for allowing us to borrow the Openstack cloud of Cloudtop
- Lastly, the people who make me happy

- Jethro Limjoco

I would like to thank my family, friends, and especially God for supporting me to finish our undergraduate project.

- Philip Nery

In no particular order, I would like to thank the following people:

My groupmates - **Jethro Limjoco** and **Philip Nery** - without them I would not have the willpower to work.

Our adviser and examiner - **Sir Ocampo** and **Sir Umali** - for taking the time to give us useful feedback.

Ate Emily - for taking care of my beloved animal companions, the beautiful cats and dogs that play in my grandparents' garden.

My parents- **Roselle Leah Rivera** and **Ed Sevilla**- for letting me skip family dinners to work on this project.

Joseph Michael Galero - for his unwavering support and guidance. Without him, accomplishing my contribution in this project would not have been possible.

- Mithi Sevilla

Abstract

MISTv: Multimedia, Internet, Storage, and Television

There is an increasing demand for cloud storage and video streaming. However, the data centers that host these services are prone to becoming single points of failure, and are potentially huge investments for service providers. In light of this problem, we developed MISTv, an alternative service that distributes server data, internet traffic, and energy consumption from the service providers to the users by using locality-aware DHT-based peer-to-peer mechanisms instead of the traditional client-server paradigm. We implemented a working prototype using Raspberry Pi single-board computers as peers and incorporated localization, data distribution, and security algorithms into the application. Experiments conducted on MISTv presented us the significant effects of the chunk size, mini chunk size, Kademlia constants alpha and k, and encryption on the download speed, and at the same time showed the great performance cost of secure peer-to-peer communication. Localization tests show inconclusive results with respect to the system performance due to the nature of the testbed.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Overview

Video-on-demand is a rapidly growing service used by people around the globe. From 2008 to 2012 the number of adult internet users in the U.S. that watch full-length television shows online has increased from 49.6 million to 72.2 million [3]. Over six billion hours worth of video are watched each month on YouTube alone [4]. In 2013, Sandvine reports that video streaming accounts for 78% of all downstream traffic in terms of bandwidth and it is projected that in 2017, more than 90% of traffic shares will be online video [5].

Aside from the increasing demand in video streaming, there is also an increasing demand for storage. According to the Gartner Survey, 47% of enterprises rank data growth in their top challenges and 62% of responders reported that they will be expanding the capacity of existing data centers in the year 2011 [6]. Dropbox, one of the leaders in providing cloud storage, has around 50 million users in 2011, and it continues to increase every second [7].

With the increasing demand in data storage and video streaming services, the costs and issues of data centers cannot be ignored. Energy-consuming data centers have high start-up and maintenance costs and are prone to becoming single points of failure. In the year 2013 alone, Google invested 1.6 billion USD on data centers in a span of three months [8]. Gartner even estimated that the global expenditure for data centers will hit 143 billion USD by the end of this year [8]. Also, The average cost of server downtime is 300,000 USD an hour and each year more than 26.5 billion USD is lost in revenue due to server failure [9, 10].

Decentralization is a possible solution to meet the demands of the users and at the same time address the problems of data centers. The idea of decentralization is to distribute the data, traffic load, and power consumption to the users. This concept can serve to reduce the problem of expensive server maintenance costs and server down-times. In order to decentralize video streaming and file management, we need to use a network architecture that does not completely depend on servers. The peer-to-peer architecture has the mechanisms to act with minimal server interaction.

Peer-to-peer networks also have the property of increasing file download speeds and video streaming rate if there are a good number of copies of those files over the network. One of the main problems of peer-to-peer mechanisms, however, is that users tend to turn off their computers when not in use, which reduces the number of sources for the files that they have. Also, internet service providers block or throttle peer-to-peer based traffic because inter-autonomous system traffic is costly [11].

We developed a low-cost system for video streaming and cloud storage which uses advanced peer-to-peer network mechanisms.

Unlike ordinary peer-to-peer services, MISTv prioritizes peers within the same network cluster as the user. With some modifications, ISP's can save money and bandwidth from traffic going outside its own domain with our system. MISTv is also resistant to churn as it is a separate device directly connected to a router.

Since our devices will store personal data and media files, we ensured that the system will not be compromised. We also developed an initial permission scheme system to protect the digital rights of the users and the providers.

1.2 Flow of the Project

We take a look at research that discuss the elements of our project, namely, video and localization, distributed storage, and security, in order to determine existing subproblems in our project, as well as concepts we can integrate into our project. Next, we formalize the problems that we want to address and the goals of our project. Then, we discuss how we built our project and its components, and the engineering decisions we made along the way. We then analyze the behavior of

our system under different system conditions, and the effects of localization on our system. Finally, we present our conclusions for our project, and recommendations for future researchers who wish to work on our project.

Chapter 2

Review of Related Work

We studied various works to understand, develop, and synthesize the following elements of MISTv: (1) video streaming and localization, (2) distributed storage, and (3) security. This was done in order to create a service that combines multiple concepts that it needs to function properly.

MISTv is a peer-to-peer based service that provides secure video streaming and distributed file storage solutions in order to reduce the costs of server maintenance and losses due to single points of failure. It is also able to reduce the traffic between network clusters, and at the same time it ensures fast and efficient downloads. Also, we provided an efficient distributed storage mechanism, which includes fast resource location, persistence of information, load balancing, caching, maintenance of directories, and permanent data storage. Finally, security features were implemented in the service in order to prevent malicious attacks to the system, ensure the privacy of the users, and protect the digital rights of the providers.

2.1 Video and Localization

An important consideration for a P2P video streaming service is to reduce inter-autonomous system (inter-AS) traffic. Because peering decisions in P2P are essentially random, P2P systems have significantly increased the expenses of ISPs. With localization this will be mitigated.

The principal goal of localization is to reduce inter-AS traffic. Simulations have shown that it could be done without affecting performance [12]. In fact, it could even increase the download speed of a P2P video streaming system [13]. However, these results are hard to replicate in practice. Localization can have limited impact and it can even have reduced performance and robustness [14] as local peers may be scarce or may not have large upload bandwidth capabilities

[15, 16].

These attempts in applying localization to P2P are based on BitTorrent as it is the most widely used P2P protocol. However, P2P was not originally designed for on-demand video streaming applications. Because MISTv's P2P protocol does not use BitTorrent's protocols, the problems and limitations inherent in BitTorrent are avoided [17]. In addition, because MISTv is expected to be frequently up and running, peer availability would be better. With peer availability comes greater benefits from localization.

This paper presents a prototype scheme for localization that assigns an AS Number (ASN) to each peer. These ASN's serve to group peers into network clusters. The ASN is part of the data returned when looking for peers. The node will only download from peers with the same ASN. If no peer has a matching ASN, this feature is ingored.

2.2 Distributed Storage

The use of a well-designed peer-to-peer storage becomes most relevant in the light of the increased demand in storage and the losses due to single points of failure. One model used for evaluating peer-to-peer storage is the distributed memory model which partitions the nodes of a network into three parts which are (a) the directory for routing (b) caching and (c) where data is permanently stored [18].

According to experiments, the average number of hops and the average distance between any given node is smallest when the memory architecture contains all of the aforementioned components [18]. Also as memory pressure - the sum of the size of unique data divided by the total available storage size - increases, caching becomes increasingly important as cache-less systems perform the poorest at high memory pressure. It is important to note though that the experiments were based only on simulations [18].

Permanent data storage on the other hand is important so as to ensure that the data is not overwritten by more popular data in the cache. A directory is needed to ensure efficient routing in search and retrieval. The storage system we implemented contains the three partitions and we worked on adjusting the relative size of the parts which can provide real-world evaluations of the system architectures based on this model.

Designing a storage system for a peer-to-peer network has to consider properties of a peer-to-peer system and its effects in performance including, but not limited to, fast resource location, load balancing, persistence of information, and background communication costs [19]. Many proposed systems attempt to address these design issues. Some notable examples are FreeNet, PAST, KELIPS, and BitTorrent, which will be discussed in the next few paragraphs.

Persistence of information is directly and indirectly achieved by BitTorrent [20] by employing principles in its exchanges such as a "tit-for-tat" policy, where a peer only sends a chunk of a file to another peer when that peer gets something in return, and a "rarest-first" policy, where the rarest chunk is given priority. "Tit-for-tat" will prevent users from downloading files without contributing to the system, and "rarest-first" ensures that these chunks will not continue to stay rare, encouraging more availability of data. Even with this is mind, BitTorrent still utilizes a traditional server-client scheme for its trackers, which makes it prone to single point of failure problems. This is not an issue in our project because our system does not need trackers.

Fast Resource Location has been done by FreeNet [21] by caching data near the requesting peer, but being a "cache-only" and an unencrypted system with a limit on the number of hops, the system "forgets" unpopular data and provides no way of deletion. This also means that stored files exist without privacy and retrieval guarantees even if the file is still in the network. Unlike this service, we provide permanent storage for user data that is encrypted, which will give the owners more control over them ensure guarantees of retrieval and privacy.

More data-availability, which provides churn resistance (resistance to the negative effects of peers coming and going), is achieved by KELIPS [22] by hashing files into groups of nodes and disseminating information through a gossip-protocol, but this is not without the cost of large memory requirements and high background communication. Node failures are similarly addressed by PAST [23] by providing permanent replications all over the system, however without a directory, the look-up is slow. In contrast, our system has a directory and performs replications without consuming a lot of memory and requiring high background communication because we did not implement the "hash-to-group" mechanism.

All of these systems are designed to be implemented in software for personal computers. This is a problem because the software clients are always turned off after use. We implemented

these storage mechanisms on a separate device that is always turned on.

Using a modified version of Kademlia [24], a DHT algorithm, to determine where to put data permanently, load-balancing and fast look-up can be improved. This is because Kademlia employs k-buckets which determines a neighbor node's reliability, and an XOR distance computation for symmetry between nodes. Persistence of information in the presence of unmanaged volunteer participants was employed via replications of the same data in different nodes. The original Kademlia algorithm does not include replication; however, Kademlia has been said to be easily adaptable to better suit enterprise networks in providing different privileges in data access as well as it being an easy and useful algorithm for the purpose of generating permanent replications due to its tree-like structure (tree-like data memorization model) [25]. These modifications in Kademlia was included for privacy and persistence of information in case of node failures.

In summary, our storage system has each node possess a directory, a cache and permanent data storage. Fast look-up, persistence of information, and load balancing was achieved by a modified version of Kademlia - which provides replication. We implemented this system on real devices that people can use for practical purposes.

2.3 Security

Another dimension of MISTv is security. Our system addresses the following problems: (1) Protection of the application's protocol from attacks of malicious users which are intended to affect the performance of the network, (2) Protection of the end user's and provider's digital rights, and (3) Decentralization of security.

It is necessary to protect the system from having its performance degraded because nobody will be willing to use a video streaming application that is slow or intermittent.

However, using the standard Kademlia DHT routing algorithm alone is not enough to prevent performance-degrading network attacks such as Sybil attacks, Eclipse attacks, and Adversarial routing [26]. These attacks exploit the fact that you can easily fake your identity and join a Kademlia network for whatever purpose [26]. There is also no mechanism to check whether a peer is malicious or not [27]. This means that we need make it difficult for malicious users to join the network and do as they please.

Security measures for Kademlia have been proposed in the literature, however none of them have implemented their algorithms in real systems [26, 25]. Our project is a modified implementation of their security measures on actual devices.

In order to prevent malicious users from easily joining the network, we implemented a mechanism that securely assigns ID's to joining peers. There are two ways to implement secure peer ID assignment. First is the use of a secure third-party host in ID assignment and verification [26], and second is the use of private encryption keys [26][28]. In the use of third-party hosts, it is necessary for service providers to invest in hosts in order to generate secure peer ID's. This is a potential problem because these hosts can become points of failure if they are found, which will demand tighter security measures for these devices. Using private keys will prevent this problem, however, it will cost having a more complex and multilevel system. Our secure node ID generation and communication system combines elements from both ideas to allow a degree of control over the network, and at the same time reduce the need to have tight security measures on our third-party hosts.

Kademlia also uses sequential search [24], meaning that it searches for files one path at a time, which is slow and prevents the protocol from finding dead nodes sooner. It can also be exploited by malicious nodes because they can redirect the traffic to dead ends [26]. In order to prevent this exploit in affecting the performance of our system, we incorporated parallel search into the protocol. This means that multiple searches for target files across different starting points is done by our modified version of Kademlia. We also determined the optimum number of parallel searches to be done at a time while considering processing overhead and search speed at the same time.

Users have the right to have their personal files protected from being viewed and stolen. Video providers also have the right to prevent their paid videos from being watched for free as that is piracy. Since our application deals with file storage and video-on-demand, we added features to protect the digital rights of the users. In order to uphold digital rights, we added identifiers to files in order to classify them as files belonging to a specific user, files that are shared to a limited number of peers, and files that can be shared to anybody. This idea stems from a concept of adding identifiers to peers in order to determine their hierarchy, which limits peer interaction depending on the peer's hierarchy for security purposes [25]. On the other hand, our project uses

the concept of identifiers in order to allow or prevent users from gaining access to specific files depending on the permissions that they possess. This is to prevent illegitimate and legitimate users alike from accessing files that they should not be allowed to access, which is a first among security implementations for Kademlia-based systems.

Chapter 3

Problem Statement and Objectives

3.1 Problem Statement

Data centers hosting services that address the increasing demands for cloud storage and video streaming are (1) prone to having single points of failure, and (2) potentially huge investments for service providers. Decentralization has been a proposed alternative to this problem, and many peer-to-peer systems were designed in this light. However decentralized systems have issues of their own, including (1) high inter-ISP communication costs, (2) data-availability and fast retrieval in the presence of unmanaged resource participants, and (3) privacy and protection of the network and its users from malicious attacks. Algorithms have been designed to address the aforementioned issues such as localization, modifications to existing DHT algorithms, encryptions and peer id assignments. We present a prototype that brings together all of these ideas in the light of providing an alternative solution in meeting the demands for video streaming and data storage.

3.2 Objectives

We aim to develop the service such that it will (A) use a DHT-based peer-to-peer network for storing and retrieving data, (B) enable a free and open-source media player to seamlessly and sequentially play chunks of videos taken from different peers (C) create and use an encryption system for the files stored in the device so that only those who have permission can access the said files.

A Kademlia-based ISP/locality-aware peer-to-peer DHT protocol is implemented in the service. The application was tested by having the users store and retrieve data to and from different

nodes in the system. For security measures, we implemented the theoretical security features for Kademlia-based systems such that it will prevent the following: (A) malicious users from hijacking users through the application by following good programming practices and standards, (B) malicious users from finding and hijacking critical nodes by adding more layers of security to these nodes and (C) semi-malicious nodes from exploiting the network by using well-defined file identifiers.

We implemented prototypes for MISTv using Raspberry Pi devices, which are cheap and low-energy credit-card-sized single board computers, to serve as a proof of concept.

Chapter 4

Methodology

4.1 Overview

4.1.1 Flow of the Project

First, we created our own base Kademlia code based on existing Kademlia code. Then, we modified the Kademlia code such that it incorporated replications, caching, file permissions, secure communication, and encryption. We also created an add-on for an existing open-source media player so that we can download and stream videos. An admin panel that runs on a local web server is used to control an individual Raspberry Pi via a web browser.

4.1.2 Network Architecture

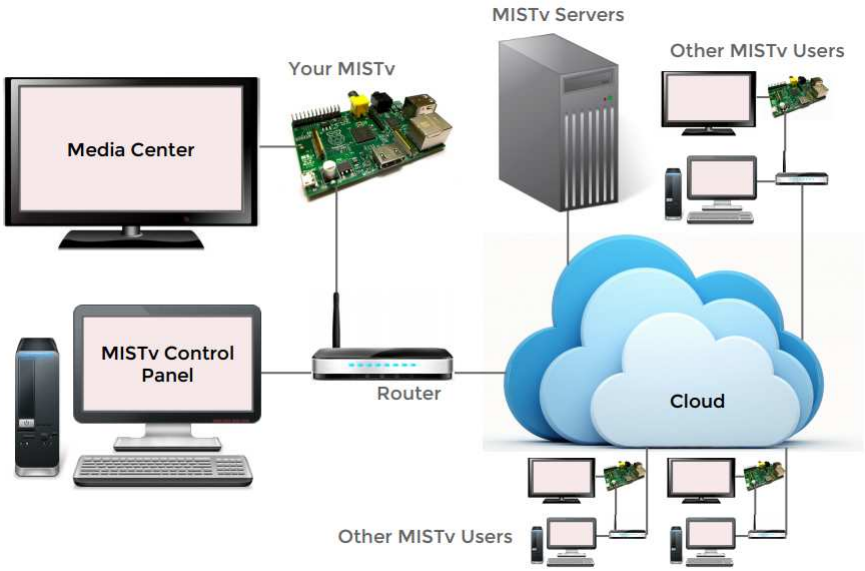

Figure 4.1: MISTv's Network Architecture

Figure 4.1 illustrates MISTv's Network Architecture. The user blocks User#1 and User#2 represent other MISTv clients. The MISTv server block then represents a MISTv server that is separate from the users, but MISTv servers are not central to the network. This means that these servers can fail or disappear without affecting system performance considerably. The purpose of these servers in the network is that they aid in security and backup, and they also serve as the source of video files from providers. The MISTv and router block represent a view of a single user's system. A desktop system is used to control and interact with a personal MISTv, while the Media Center (e.g. a TV) is used for watching videos.

4.1.3 Protocol Stack

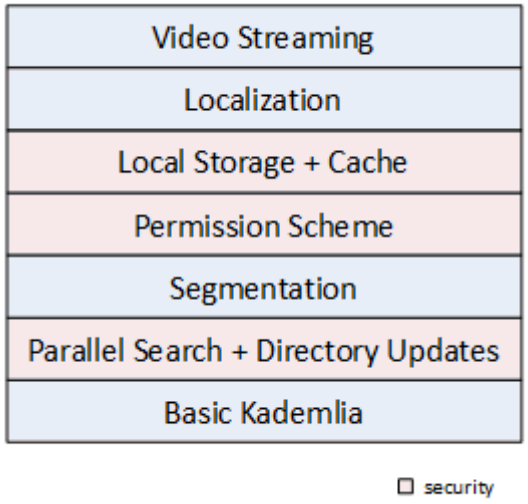

Figure 4.2: MISTv's Protocol Stack for the MISTv Program

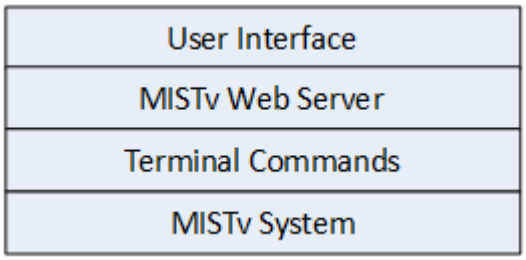

Figure 4.3: MISTv's Abstraction Layers

Figure 4.2 is a representation of the protocol stack of the program we created for MISTv. Everything was put on top of the Basic Kademlia code we made. Segmentation, local storage, caching, localization, parallel search, and directory updates was implemented as part of the algorithm for the overlay network. The video player is able to play all peer-to-peer video streams that communicate with our program. All blocks colored in pink are layers that have security features integrated into them or are security features themselves.

Figure 4.3 then shows the layers of abstraction for our application. The MISTv system is the one performing all the algorithms and functions. These functions are triggered by the terminal commands. Then, a MISTv Web Server is used to easily control the devices using a browser.

Finally, a user interface allows end users to easily use and interact with the appliance.

In order to implement the algorithms for the overlay network, we first modified Kademlia such that it will use parallel search instead of sequential search for greater efficiency and to secure the overlay network from adversarial peers. Then, files to be distributed over the network are segmented into chunks before transmission. These segments are associated with access permissions to allow users to restrict access to limited peers. Then, these segments are securely stored in the user's local directory if desired. Segments from other users are also stored in the user's cache. Then, all MISTv traffic is localized. Lastly, video files can be streamed to a media player.

On the filesystem side of our application, we first created a directory where MISTv user files can be stored. All files have a user-defined access permission to restrict visibility and access to their personal files. Next, we created a terminal-based interface to interact with the entire program for testing and debugging purposes. Then, we modified a selected video player for it to play peer-to-peer video. Finally, we created a web-based user-friendly user interface for our product to be usable.

4.2 Media Player and Operating System

4.2.1 Raspberry Pi

We used six (6) Raspberry Pi machines connected to a Local Area Network. A snippet of the set-up is shown below in Figure 4.4. In order to view the images being displayed by the Pi, we used a widescreen television to connect the HDMI port of the Pi.

We prototyped our project on six (6) Raspberry Pi devices connected to a Local Area Network. We chose to prototype under the Raspberry Pi platform because it is a credit-card sized networking capable computer that costs 35 USD (approximately 1500 PHP) [29], which is a sufficiently cheap platform for prototyping our project.

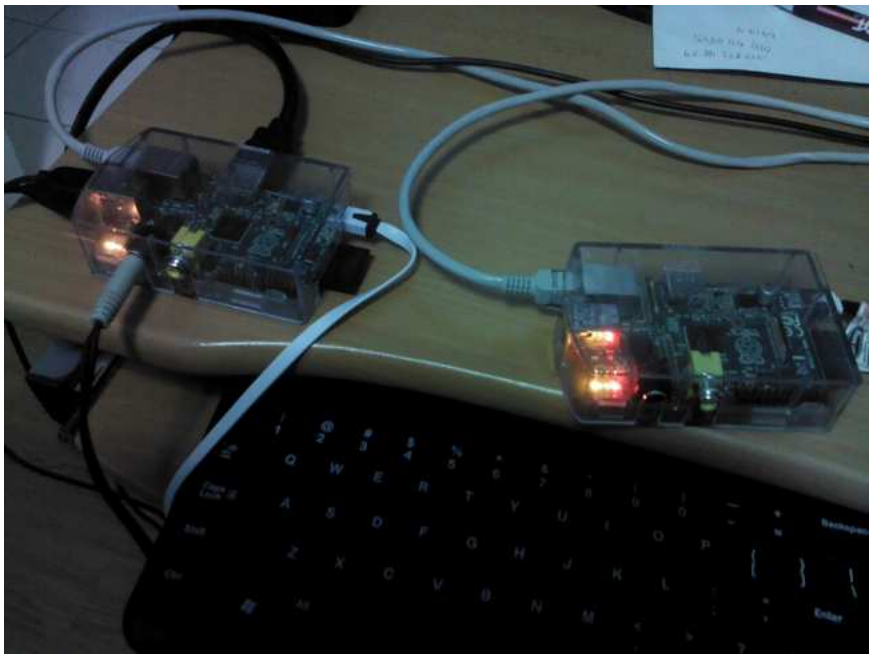

Figure 4.4: Raspberry Pi Machines connected to a LAN

4.2.2 Selection of the Operating System

OMXPlayer, the default video player in one of the most popular Operating Systems for the Raspberry Pi, Raspbian Wheezy [30, 31], is insufficient for our application. Based on our tests, we experienced frequent video and audio lag while playing high-definition videos. It also lacks the familiar user interface in home theatre software. Two viable alternatives supported in the Raspberry Pi are XBMC [32] and Plex [33]. The two players are usually installed in the Raspberry Pi by using distros that are pre-installed with them. The three most popular media center distros for XBMC are OpenELEC [34], Raspbmc [35], and Xbian [36] while the only distro for Plex in the Raspberry Pi is RasPlex [37]. RasPlex is a new project and so there is not much activity in the community therefore we ruled out RasPlex. OpenELEC is harder to install and it does not leave a lot of room for modification so we also ruled it out [38]. We tested Raspbmc but playing videos and moving through the interface was sluggish and so we also ruled it out. We tested Xbian and because it was faster than Raspbmc, we chose it as our media center distro. We also chose Xbian because by using Xbian, we can already control the Television (TV) using a remote control, provided that the TV is Consumer Electronics Control (CEC) compliant [39].

4.2.3 Use of Virtual Machines

For us to conduct better tests with our distributed system, we needed more nodes than the Raspberry Pi machines we had. We decided to set-up virtual machines in order to have more nodes.

We attempted to emulate Xbian on QEMU so that we can have more machines in our testbed. Using the Raspberry Pi network setup in Section 4.2.1, and files from a github repository that contains images that promised to allow Xbian emulation on QEMU [40], we attempted to emulate the Operating System.

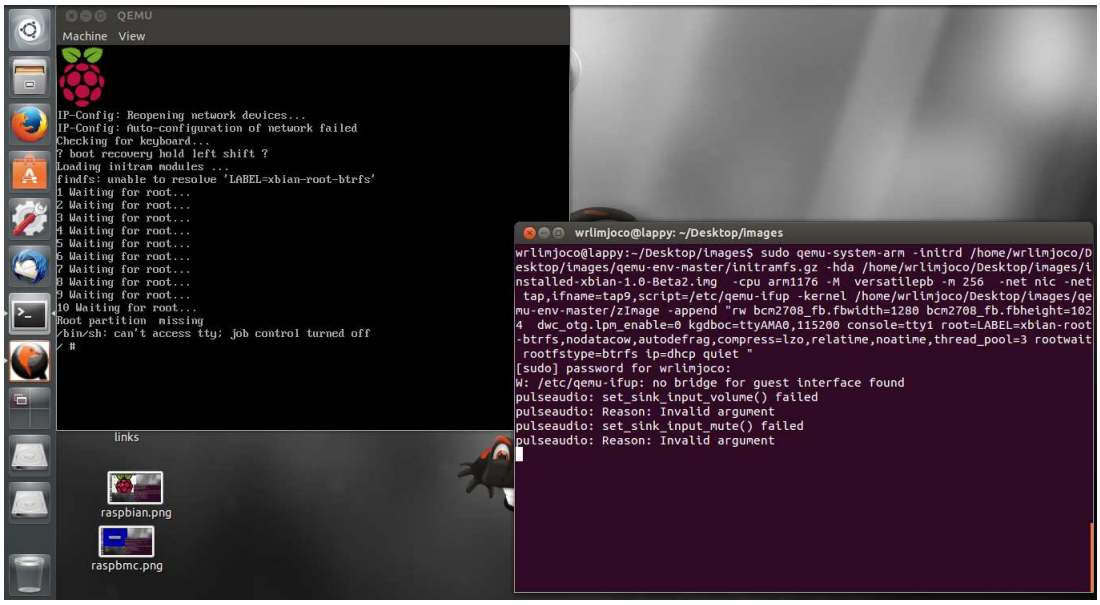

Figure 4.5: Screenshot of Attempted Emulation of Xbian

The attempt to emulate Xbian failed as shown in Figure 4.5. In order to make Xbian work in the QEMU environment, much kernel hacking needs to be done, and that will take too much time. Then, we thought that it is not necessary for the Operating System of the emulated machines to be exactly the same as the one that we will use on the Raspberry Pi because they will just serve as extra nodes that will not play any videos at all. However, we want the emulated operating system to be as close to Xbian as possible to have a more accurate model of the entire system of our project. So, we decided to attempt emulating RaspBMC. In order to do that, we just followed an emulation guide that also has a custom kernel for the emulated system [41].

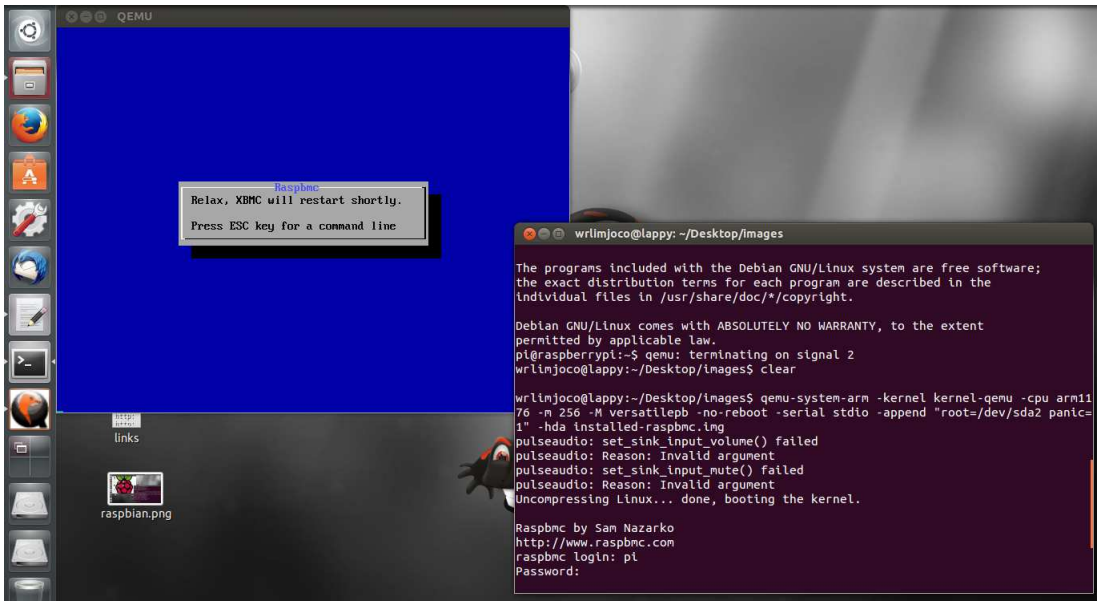

Figure 4.6: Screenshot of Attempted Emulation of RaspBMC

The results of the RaspBMC emulation are shown in Figure 4.6. QEMU is able to emulate the Operating System, but XBMC does not work properly. So we decided to emulate Raspbian OS on QEMU on six (6) desktop computers since it is the most stable among the three choices, as shown in Figure 4.7. We can emulate 2 Raspbian instances per PC, and we can also connect the host machines to the network by creating two bridge interfaces per PC. Having more than two emulated machines in one PC causes each machine to lag so much, and it makes the bridge interfaces unstable. The problem with this emulation method is that it has a RAM restriction of 256MB, which is half of the RAM of our Raspberry Pi machines. This led us to borrow an Openstack cloud from the CloudTop project.

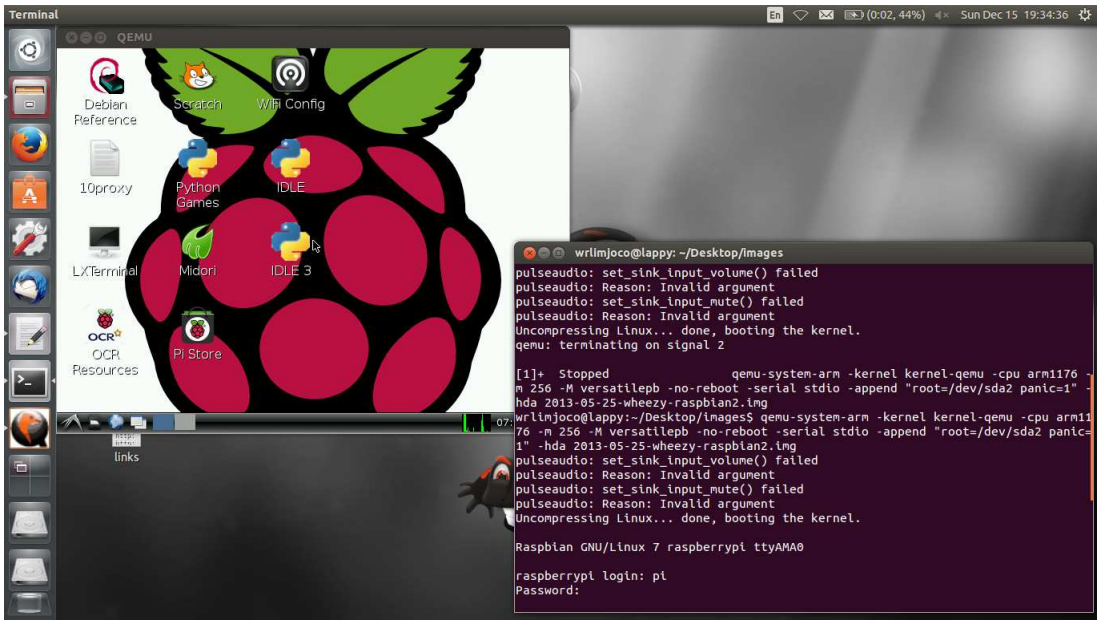

Figure 4.7: Screenshot of Attempted Emulation of Raspbian

Using the Openstack cloud we borrowed, we were able to emulate 100 machines with the same specifications as our actual devices. However, we used a 32-bit Ubuntu 12.04 cloud image instead of a Raspberry Pi image due to the inability of the cloud to emulate those images. This is still a suitable replacement because Ubuntu and the Raspberry Pi images are both Debian-based, so the differences between them are minor. The servers running the Openstack cloud is shown in Figure 4.8 below. The cloud is connected to the Raspberry Pi's shown in Figure 4.4.

Figure 4.8: Servers running Ubuntu Cloud Images on Openstack

4.3 Selection of Kademlia Code

As discussed in Section 2.2, we used the Kademlia DHT algorithm as a base for our project, and we used Python to program the entire project for the sake of our own personal convenience. Now, in order select a good Python implementation of Kademlia to use as the base code for MISTv, several factors must be considered including, but not limited to the following: (a) simplicity - it must be easy to understand, (b) compatibility - it is easy to modify with respect to our project, and (c) efficiency - the code must have been designed such that files would be downloaded and uploaded to and from other peers in a reasonable amount of time.

The top four libraries we have considered is Entangled [42], Khashmir [43], Isaac Zafuta's PyDHT [44], and Nightmare [45].

The problem with Nightmare is that it is not easily comprehensible because the code is not well-modularized. It also uses sqlite3 to store information about each peer in the network, which is not a scalable method of storing data.

Entangled is a bad idea as well because it has many bugs and, despite it being the most maintained code among the four choices, has a lot of dependencies that are not only confusing, but

are bloated as well. It has many features and complexities that we don't need, and it takes up a lot of space.

Khashmir on the other hand is not maintained. It uses many deprecated modules like WHRandom and Airhook, which are defunct projects.

Isaac Zafuta's PyDHT made it to the top of our list because of its clean code and modular style which made it easy to comprehend. However, it uses a primitive server handler library which is a low-level interface that will take a lot of time to implement maximum efficiency in multi-threading, forking and asynchronous handling of tasks. In a later note, we also found out that it does not return k number of nodes for the primitive function call 'nearest nodes' as recommended by Kademlia (see Appendix) which makes it inefficient. It also does not use any kind of document database for saving data which means that every time you exit the program all memory with be cleared after the session.

Because of this, we have decided to build Kademlia from scratch using PyDHT as reference.

4.4 Modifications to Kademlia

4.4.1 Use of HTTP

Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) are the two main Transport Layer Protocols of the Internet. TCP provides reliable, ordered, and error-checked delivery of data streams. UDP, on the other hand, just churns out data with minimal error checking, thus making it faster than TCP, but less reliable. Hypertext Transfer Protocol (HTTP), on the other hand, is an application layer protocol that can use any of the underlying Transport Layer Protocols.

The advantage of using a protocol that is in a much lower layer is that it allows us to have a higher level of control. We are able to manually select intelligent programming decisions like implementing multi-processing, multi-threading, and asynchronous handling of information to make the system more efficient. However, it is also much more prone to errors and is more complicated to debug. Using a protocol of a higher layer makes it easier for us to code, and it allows

us to automatically abstract the lower layers so that we have less concerns to handle. Due to time constraints and the number of elements of our project, we have decided to use HTTP for communication between devices in our system.

In order to let our code use HTTP for communication, we decided to use a Python module called "Requests". The Requests module claims to be the "HTTP for Humans", meaning that these libraries are modern and easy to use. They also said that the standard urllib2 module provides most of the HTTP capabilities you need but that the API is broken. To use urllib2 you have to dedicate an enormous amount of work and method overrides to perform even simple tasks. The use of Requests will make coding the communication system of our project easier. [46]

4.4.2 Use of HTTP Servers

Bottle [47] is a fast, simple and lightweight Web Server Gateway Interface micro web-framework for Python which has no dependencies other than the Python Standard Library and is distributed as a single module. Using this module will make it very easy for us to handle requests from other peers, and from our own device. However, Bottle runs by default in a non-threading HTTP server, which may become a performance bottleneck when the server load increases. So, we installed a multi-threaded server that Bottle can use, namely, CherryPy [48].

To test the speed of CherryPy compared to Bottle's Default Server, we used an Apache Benchmark Tool [49].

Table 4.1 and Table 4.2 present the results of the Apache Benchmark Test.

Table 4.1: Load Testing Without Concurrency

	Time taken for tests (sec)	Time per request on average (msec)	Requests per second on average (requests/sec)	Transfer rate (kbytes/sec)
With CherryPy	4.798	0.48	2084.03	313.42
Without CherryPy	10.057	1.006	994.38	159.26

Table 4.2: Load Testing With Concurrency

	Time taken for tests (sec)	Time per request on average (msec)	Time per request on average across all concurrent requests (msec)	Requests per second on average (requests/sec)	Transfer rate (kbytes/sec)
With CherryPy	5.29	2.645	0.529	1890.22	284.27
Without CherryPy	9.46	4.730	0.946	1057.08	169.30

For load testing with concurrency greater than 5, the system without CherryPy fails to function completely. The system with CherryPy, however, fails to finish the Apache Benchmark test completely but the test does start running for a while. The figures in table 4.2 were obatined by using a concurrency equal to 5.

As you can see from the two tables above, the performance of the system significantly imporves with the use of CherryPy.

4.4.3 Database

Our project uses a database to store important node information and file information. So it is important to select a good and easy to use database for the given tasks.

Relational Databases, like sqlite3, has been the dominant model for database management, but the use of non-relational databases (like MongoDB) has been steadily increasing because of its advantages. Non-relational Databases are scalable compared to relational databases. Volumes of large data can be handled faster by Non-relational Databases rather than Relational Databases. Non-relational Databases are flexible and elastic as they does not have a scheme, and they are cheaper and more economic because of this in the long run. [50]

We have decided to use MongoDB because of this, instead of sqlite3. Also, although regular CSV is also scalable, it is harder to sort and find documents with it.

We used a tutorial [51] to install and learn how to used MongoDB using PyMongo.

4.4.4 Node Information

4.4.4.1 MISTv Bootstrap Server

Figure 4.9: Laptop serving as the bootstrap server.

A MISTv Bootstrap Server is a node that contains a list of registered nodes, a list of file id's and their corresponding descriptions, and the public keys of the registered nodes. The bootstrap server can be a laptop or a powerful CPU as show in Figure 4.9 above because it serves as a public database to aid the functionality of MISTv. Registered nodes send the following information to the Bootstrap Server for storage in its database: node ID, server, port, public key file, database index, and time last updated. A MISTv Bootstrap Server sends information about a random registered node currently in its database to a node that just registered to it so that newly joined node can populate its contact list using the registered node as its own contact. This is done by pinging the server and storing information to that server (See Appendix B.3). Each time a registered node uploads a file in the MISTv network, it also sends information about the file in a MISTv Bootstrap Server. This information can be queried by a user node when it is looking for a particular file in its network that it wants to download. The Bootstrap Servers also store file ID, file title, file description, and file size information. To register a file to the server, we use a

function called register_file() to tell the server these pieces of information. Then, register_file() calls a function called files_upsert() to store these pieces of information in the server database.

4.4.4.2 Regular User Server

Each node stores information about its own files, information about the nodes in that it knows of, and information about the files that is stored in the node as determined by the Kademlia DHT protocol (See Appendix A). The server is informed when new data is available by using a function called update_file_data(). The file information stored in your own server are the following: file ID, file title, file description, file extension, file type, storage type, permission type, permission list, locally stored flag, searchable flag, encrypted flag, owner's node ID, owner's port used, and owner's own regular user server. Two additional pieces of information, access request list, location list, are included in the file information if the file does not belong to you.

4.4.5 Caching

Caching allows nodes to temporarily store files or chunks of files they have downloaded but not stored locally in the Raspberry Pi for quicker subsequent access and use. The cache also takes note of the time the stored file or chunk was last accessed. This is done by allocating a cache size and a local storage size based on the total capacity of the Raspberry Pi. Files or chunks tagged for caching cannot exceed the cache size limit. Locally stored files also cannot exceed the maximum allocated size for local storage. This means that you cannot proceed to add anything to the local storage if it is full, unless you delete files in the storage. The cache system replaces old cached files with new cached files by replacing the least recently accessed file.

4.5 Basic Messages and Terminal Commands

The basic Kademlia DHT (See Appendix A) consists of a directory (known as k-buckets), four basic messages that nodes send to each other and four iterative user commands that use the basic messages as building blocks. The basic messages are store, find_value, find_closest_nodes, and ping, These are used for nodes to (1) look for particular nodes, (2) store, (3) search for particular files, and (4) join the network.

In addition, these basic messages are also used to make sure that the directory is up to date. We also included three more basic messages which are delete, update, and request_access in order to remove files users want to delete, and to change information concerning files such as permission access and searchability.

There are ten basic user commands. Each command is discussed below in Table 4.3.

Table 4.3: Terminal Commands

Command	Description
dht_permit(file_id, permitted_id)	Permit permitted_id to access file_id
dht_request_access(file_id)	Request access to file_id
dht_remove_access(file_id, permitted_id)	Removes access rights of permitted_id to file_id
update_directory()	Adds more nodes to your bucket list
get_my_file_data(file_id)	Allows users to view file information of file_id
join_network()	Allows users to join the Kademlia network
dht_update_file_data(my_file_data)	Updates file information
dht_store(path_to_store, file_data)	Stores file and related information to path_to_store
dht_open(file_id)	Opens file_id
dht_delete(file_id)	Deletes file_id

The variables file_data and my_file data used by dht_update_file_data() and dht_store() are dictionary variable that contains the following information: file path with file name and extension, file title, file description, file size, permission type, searchable flag, and locally stored flag.

These user terminal commands are not used directly. Instead, a web page (discussed in

section 4.6.2) that can be accessed by a user's laptop uses these commands via a Graphical User Interface (GUI) to make the service more usable to average users. The code used to build the webserver can be found at Appendix B.4.

4.6 User Interface and Video Playback

4.6.1 Video Playback

Initially, a bootstrap server was set-up to host the file list. Nodes that intend to share their files will need to register their files to the bootstrap server. The filename, the file size, the chunk size, and the nodes sharing in the network are stored in the bootstrap server. Since the bootstrap server holds these data, the nodes need only to query the bootstrap server to list all the files in the network and to list the nodes sharing a particular file.

An XBMC add-on written in Python was written to enable the media player to play video files in the network. Upon opening the add-on, it will query the bootstrap server for a list of video files. After selecting a video file, it will then query the bootstrap server for the nodes currently sharing it. The add-on starts downloading the file chunk by chunk and prioritizing the nodes with the same Autonomous System Number or ASN as it if permitted to do so. If the user has no access rights to the file, it will block the user from streaming the file. A message is also displayed informing the user that they do not have access rights to the file and that a request to access the file has been sent to the owner of the file. The ASN simulates which nodes are in the same ISP. The download works by downloading chunks of the file in mini chunks of adjustable size not exceeding the chunk size in order to show users a more granular progress bar (See Appendix B.1 and B.2). After buffering the video, the add-on can now commence in playing the video.

4.6.2 Control Panel

A web page hosted on a local web server will serve as the User Interface so that users can easily use the terminal commands discussed in Section 4.5 from a web browser for their convenience. This local web server was set up using Bottle which is a fast and lightweight WSGI micro web-framework for Python [47]. It is then set up to use CherryPy as its server backend to increase its performance as Bottle is inherently a single-threaded server [48]. Initially, once the users have

logged in, the free space left and the files ready to be shared are displayed.

4.7 Segmentation

Files in MISTv are segmented before distribution in order to reduce the processing load for each node, and to distribute the files evenly across the network in order to balance the load of each node.

Segmentation has two parts, namely, storage and retrieval. These two aspects are discussed in the following subsections.

4.7.1 Storage

For the segmentation storage protocol, if a file exceeds a certain adjustable size, that file will be chopped into chunks, and each will be given a file ID. Each chunk is stored at its respective node using Kademlia's iterative find function algorithm as discussed in Appendix A. A master ID with a database containing a map of the order of a chunk and its respective file ID is stored at the network using the same iterative find function algorithm.

4.7.2 Retrieval

For the segmentation join protocol, the file data of the master ID is retrieved using the iterative find function algorithm, which includes the database of chunk ID's and its position in the file. Each chunk is then retrieved using the iterative find function algorithm as well. Each chunk is then sewn together in the proper sequence. If one chunk cannot be retrieved, it will return an error that not all chunks can be found in the network.

4.8 Localization

We assigned ASN's to the various nodes to serve as a locality number. This locality number groups nodes together, clustering them into pseudo ISP's, since we only have a testbed that works on the local area network alone as discussed in Section 4.2. Our system looks for segments using their file ID and iteratively searches for that file ID in the nodes in its bucket list. Localization

allows the node to only download from other nodes with the same locality number, the other nodes with the file ID in their databases are ignored. If there is no peer with a matching locality number, the normal iterative search for the file is performed. Communication without localization is shown in Figure 4.10 below. Communication with localization is shown in Figure 4.11 below.

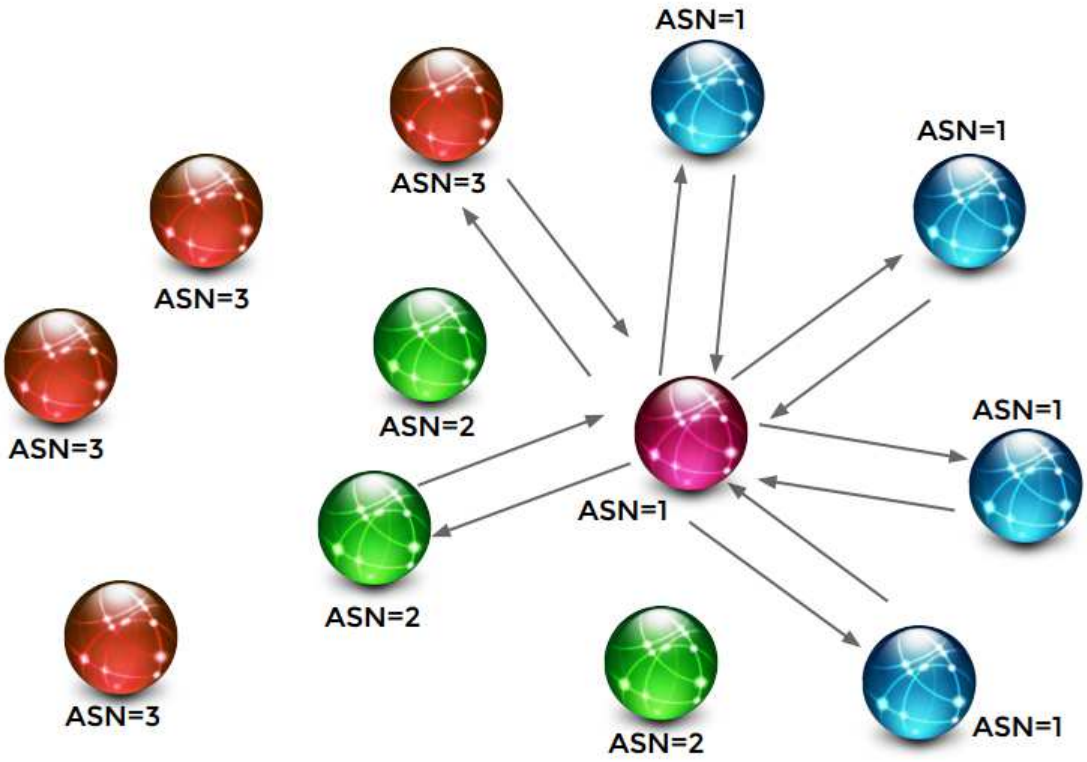

Figure 4.10: Visual Representation of Communication without Localization

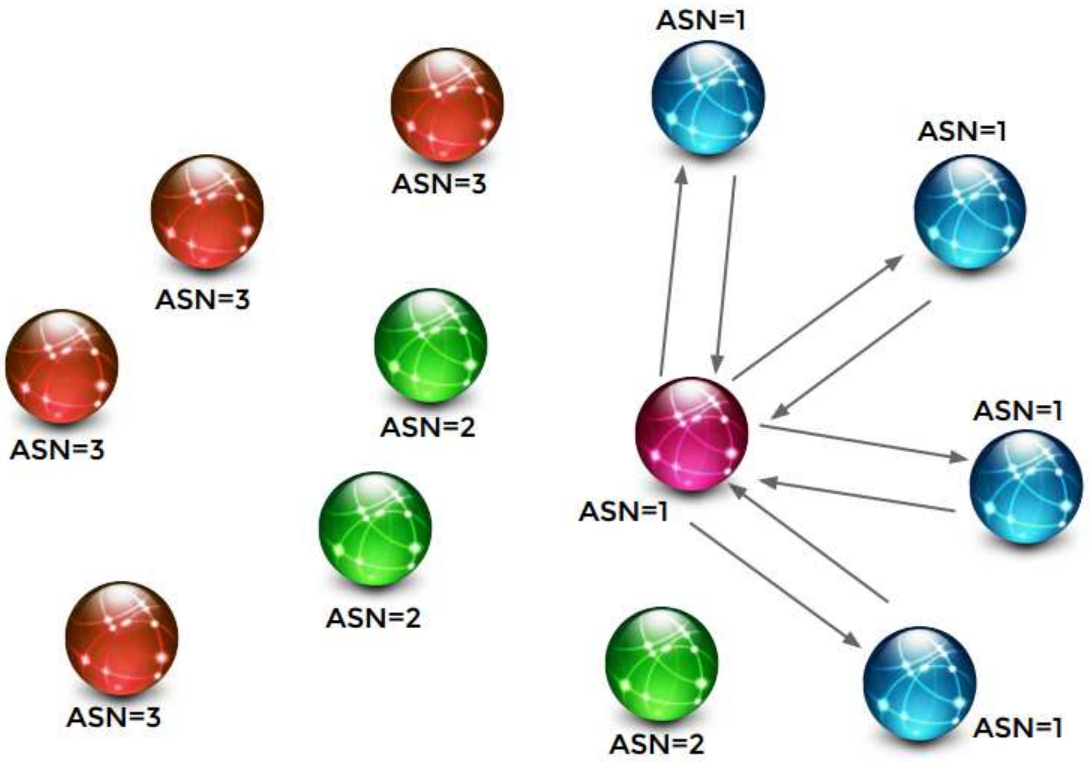

Figure 4.11: Visual Representation of Communication with Localization

4.9 Security Features

4.9.1 Secure Node ID Generation and Secure Communication

In order to prevent attacks against the Kademlia network as discussed in section 2.3, we developed a method to prevent malicious users from faking their identity, stealing data, and compromising the network. In this system, let us say that we have two people, Alice and Bob, that want to speak with each other. However, there is an eavesdropper, Eve, that is intent on tricking Bob to believe that she is Alice so that she can send malicious messages, or so that she can prevent Bob from receiving any messages, or so she can steal data from the communicating parties. In order to prevent these things from happening, there needs to be a way for Bob to verify that he is indeed talking to Alice, and not an eavesdropper like Eve. There also needs to be a way to prevent Eve from knowing what the two parties are talking about for her own benefit.

We use a encryption and digital signature system to prevent eavesdropper peers from per-

forming such an attack [52]. The security modules used to build the Secure Node ID communication system are from a Python module called python-crypto [53]. This security system is illustrated in Figures 4.12 and 4.13 below.

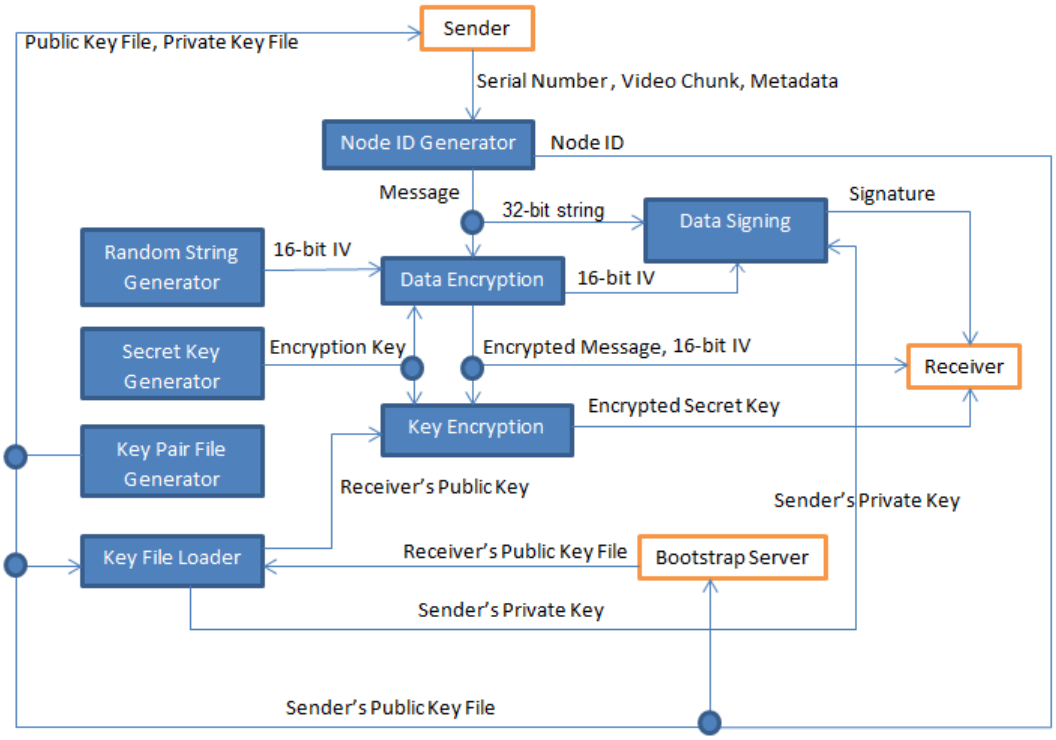

Figure 4.12: Secure Node ID Generation and Secure Communication System - Sender Side

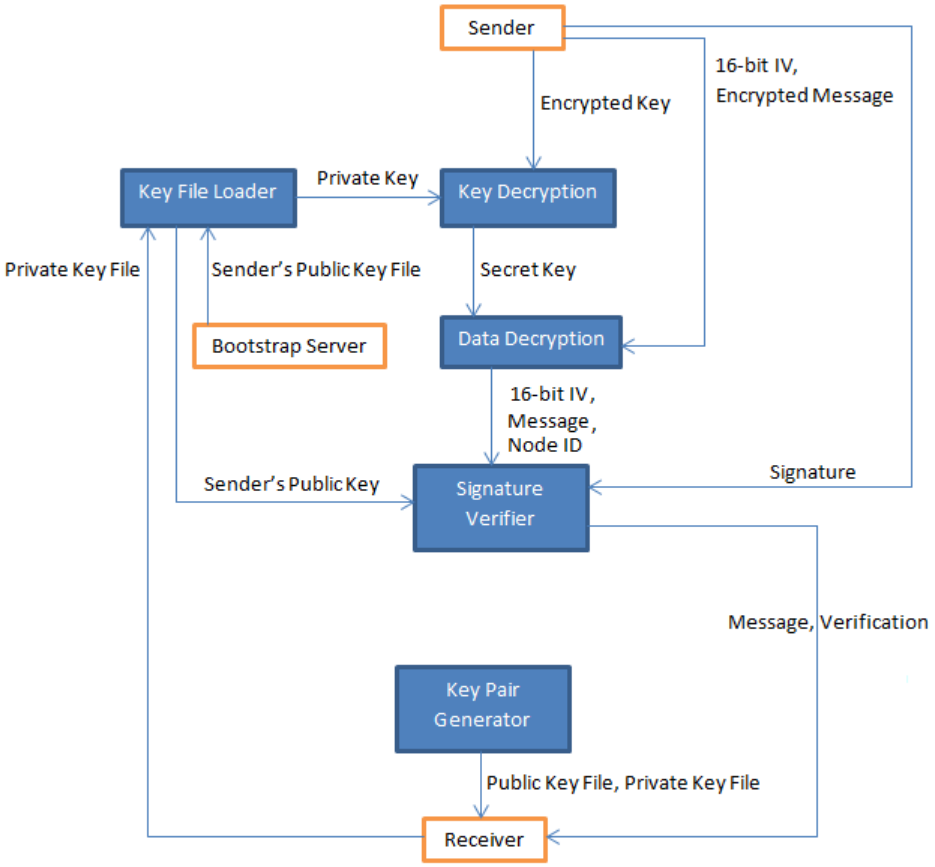

Figure 4.13: Secure Node ID Generation and Secure Communication System - Receiver Side

Figure 4.12 above illustrates the designed secure communication system on the sender's side. It uses a Node ID Generator, which hashes the input unique serial number of the sending user's Raspberry Pi to use it as a unique Node ID. The serial number is hashed to ensure that Node ID replication will be difficult so as to prevent anyone from easily generating a fake Node ID to steal the user's identity. This Node ID, along with the message the sender wants to pass on to the receiver (in our case, chunks of data, video or otherwise), is then encrypted using the AES Symmetric Key Encryption algorithm coupled with the PBKDF2 Secret Key Generation standard, which was used because its use provides a sufficiently fast and secure data encryption algorithm [52][54]. Then, the Secret Key required to decrypt the encrypted message is encrypted using the RSA Asymmetric Key Encryption algorithm coupled with PKCS1 OAEP standards. This is because we need to share the Secret Key to the receiver while preventing anyone else from using that key to decrypt the secret message inside [54]. Asymmetric Key Encryption was not used right away because it is considerably slower than Symmetric Key Encryption [52]. Finally, a random 32-bit

string is hashed and signed by a Digital Signature function that follows PKCS 1.5 standards to allow the receiver to verify if the message did indeed come from the sender, thus verifying their identity. A random 32-bit string was used for the signature because the hash for the two messages with the same content will be exactly the same, which may become a potential security hole.

Figure 4.13 above now illustrates the designed secure communication system on the receiver's side. This side decrypts the Secret Key and the message using RSA and AES standards, respectively. Then, it takes the Sender's Public Key from the bootstrap server and the Digital Signature to confirm if the message received is genuine or not and if the message if from the true sender or not. This verification process is done to determine if the data received from the sender is trustworthy or not. When the data is deemed trustworthy, that is the only time the Node ID and accompanying data is processed. Otherwise, the received data is discarded and the sender is dropped.

The purpose of the bootstrap servers in the security system is to store the public keys of MISTv users associated with the number of days that the key is valid. Both the owner of the key and the bootstrap servers know the lifespan of the key, and this piece of information cannot be changed. The public key also cannot be changed while its lifespan is not yet zero. This will prevent anyone from easily changing the public key of any user.

The functions and modules used by the secure communication system can be found at Appendix B.5.1 and B.5.2. The functions used to encrypt and decrypt files are at Appedix B.5.3 and B.5.4.

4.9.2 File Permission Scheme

Each file has its own file ID to identify itself over the network. This file ID is generated by first taking the SHA256 hash of the sum of the Node ID of the owner and the MD5 hash of the file itself. As much as we would like to use SHA256 all the time, being considered a secure hash algorithm [54][55], SHA256's performance is slow even on machines more powerful than the Raspberry Pi as seen in Table 4.4 below, so we needed to use the MD5 hashing algorithm to hash files. We can then use SHA256 to hash the short hash of the file plus the node ID of the owner without problems to use as the file ID. The node ID of the owner was integrated to the file ID to add ownership to the file.

Table 4.4: Comparison of MD5 and SHA256 Hashing Time on an Intel i5-3230M CPU at 2.60GHz, 4GB RAM, and 64-bit Ubuntu OS

	Pycrypto MD5 Hashing Time (seconds)	Pycrypto SHA256 Hashing Time (seconds)
16 MB File (QEMU installer)	~2	~4
377 MB File (typical video file)	~21	~29
707 MB File (Ubuntu 12.04 ISO)	~56	~68

Knowing that hashing takes a considerable amount of time for typical video files, considering that the hash test was done on a machine more powerful than the Raspberry Pi, this will harm the performance of the system. So, we set an option in the user interface that allows us to manually input the file ID of a file so no computing needs to be done. This is only done for testing purposes considering that the Raspberry Pi is slow. This two-way file ID generation system is illustrated in Figure 4.14 below.

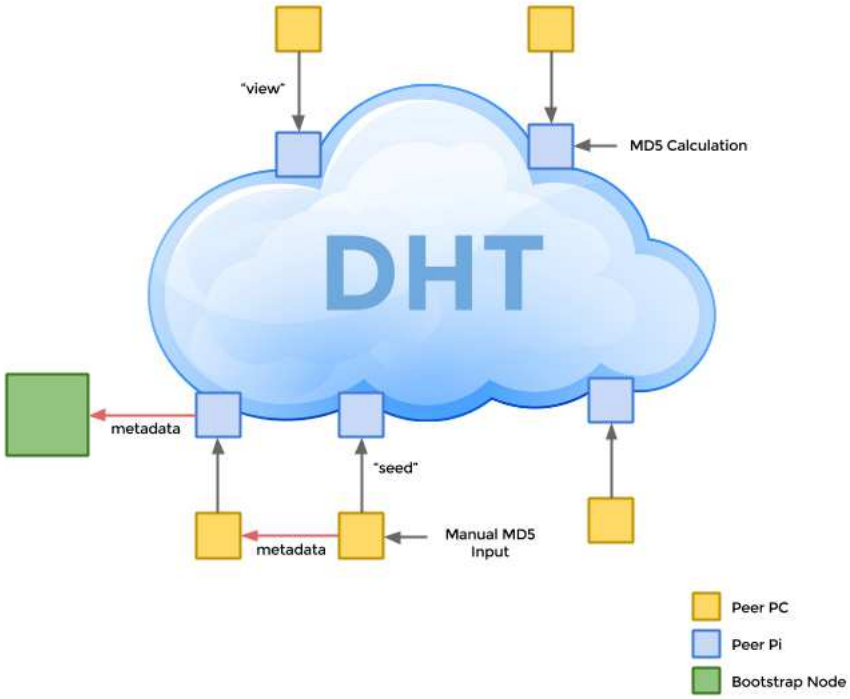

Figure 4.14: Representation of the two-way File ID Generation System

There are three types of file permissions, all, few, and me. If the file permission is set to 'all', the file is not encrypted at all and can be shared to anyone. In 'me', the file is encrypted and nobody can open the file except the owner. In 'few', the users have the ability to restrict who has access to their file. The mechanics of the 'few' permission are explained below.

Users will have the option to share their files to other users by adding approved requests of devices to a Permission Request Table attached to each file. If a user requests a file, and they are included in the Permission Request Table of a file, they are sent the AES symmetric key of that file for them to be able to decrypt the file, and the encrypted file itself if they do not have it yet. Only permitted users are allowed to decrypt these information via the secure communication system discussed in Section 4.9.1. The symmetric key is deleted after use. If a file owner decides to delete someone from the Permission Request Table of a file, the deleted user can no longer decrypt

the file even if they have a copy of the file.

4.9.3 Parallel Search

Our Kademlia code uses the original Kademlia's iterative function that searches for nodes or files, depending on the mode of operation it is run. This iterative function concurrently contacts the alpha nearest contacts in the node's k bucket list recursively until it finds the node or file it is looking for, or its contacts reply that it has reached a dead end.

The original Kademlia algorithm normally gives up searching for a node or a file when one node in its alpha nearest contacts replies that the search has reached a dead end [24]. This feature can be exploited by adversarial nodes that want to lower the performance of the Kademlia network. In order to prevent this feature from being exploited, we designed the iterative find function used by our program to keep searching for nodes or files with the rest of the alpha nearest contacts even if one contact replies that it has reached a dead end. Our modified Kademlia iterative function only gives up when all the alpha contact reach a dead end.

4.10 Download Speed Tests

In order to prove or disprove that localization will improve performance and at the same time reduce inter-ISP traffic, we tested the download speed under various conditions, which will be discussed in the following subsections. The download speed tests show the download speed of MISTv under various conditions for us to see the effects of each factor on our system's download performance.

Download speed tests were conducted over a local area network using a 16MB video file. Experiments for each test value were performed ten times, then the data recorded was averaged over those ten instaces. The data recording process was done purely using python code. The tests also show the contribution of the iterative find function to the overall performance of the system. Download speeds were computed by dividing the file size by the download time, meaning, the time taken by the iterative function is not a part of the download speed computation. All the results of these tests are presented in Chapter 5.

4.10.1 Chunk Size

This test seeks to determine the best chunk size to be used by MISTv. The assumption is that if the chunk size is bigger, the download speed will be faster because the iterative find function would have to be called less often, which means there are less chunks to find and retrieve, which should reduce the time needed to download all the chunks that constitutes a file.

4.10.2 Mini Chunk Size

This test seeks to determine the effect of the mini chunk size used to granularize the progress shown in the progress bar. The maximum size of the mini chunk size is 512 MB because it shows sufficient granularity for a 16 MB file download. We hypothesize that if we decrease the mini chunk size, the download speed will also decrease because decreasing the mini chunk size translates to needing to interact with the progress bar more often, which means that XBMC needs to query the node server and retrieve a JSON file containing the progress details more often.

4.10.3 Alpha and K

In this test, we used seven nodes and Kademlia parameters alpha and k were varied. All other changeable variables were held constant. We assume that increasing alpha, which is the number of concurrent search paths, will improve the download speed. We also expect that an increase in alpha and k will also increase the amount of time used by the iterative find function because these parameters directly control the number of iterations of the iterative find function.

4.10.4 Encryption

This test shows the effects of data encryption on the performance of the entire system. We will show the cost of encrypting your data on a peer-to-peer network of Raspberry Pi machines. We except a drop in download speed when we encrypt our data.

4.10.5 Localization

In this test, we contrasted the performance of the system with localization and without localization. We created three pseudo ISP's in the network. The delay between ISP 1 and ISP 2 is 100ms. The delay between ISP 2 and ISP 3 is 200ms. The delay between ISP 1 and ISP 3 is

50ms. No delay is added for peers in the same pseudo ISP. The simulated localization testbed is illustrated in Figure 4.15 below. The number of peers in general and the number of peers in each pseudo ISP were varied. All other changeable variables were held constant. The delays between localities were also turned on during this experiment. We expect that the delays and the number of local nodes will decrease the download speed of the system.

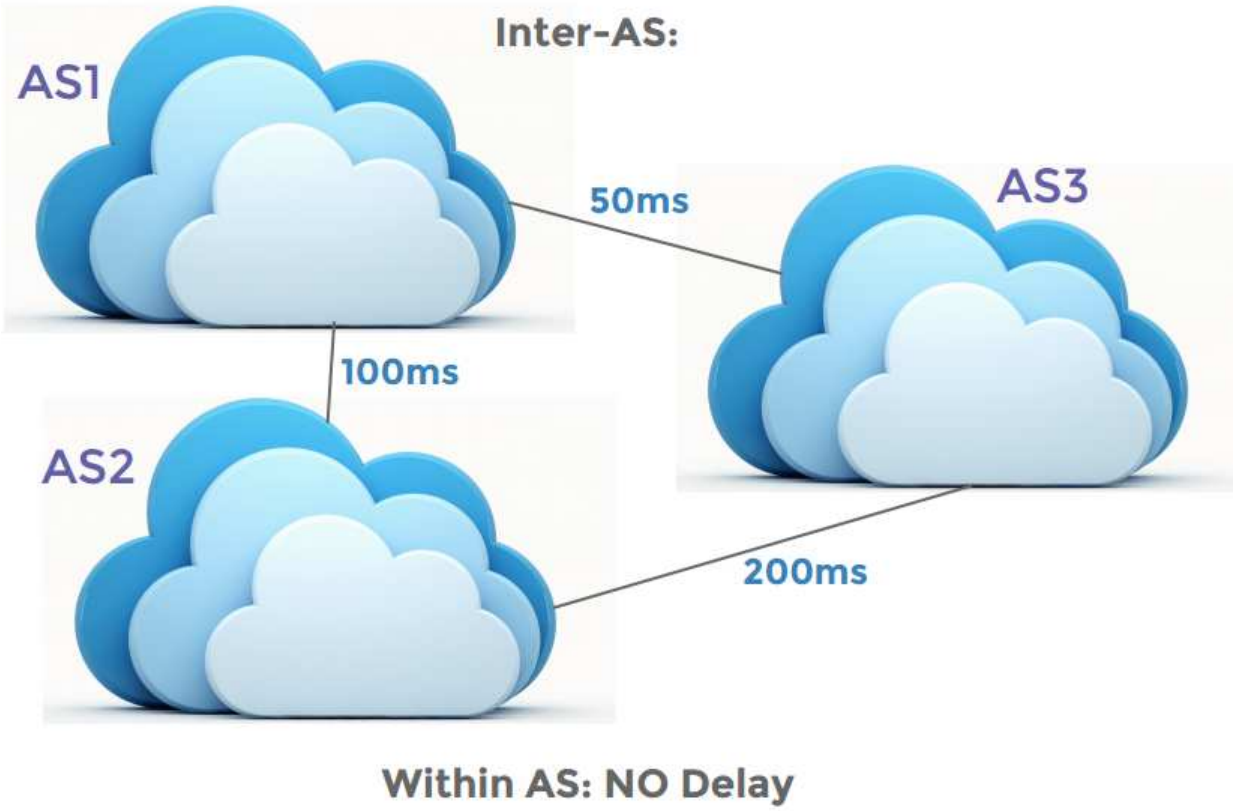

Figure 4.15: Representation of the two-way File ID Generation System

Chapter 5

Results and Discussion

5.1 User Interface Functionality

We designed a User Interface for regular users to easily interact with MISTv. Figure 5.1 to 5.7 below show some screenshots of the user interface.

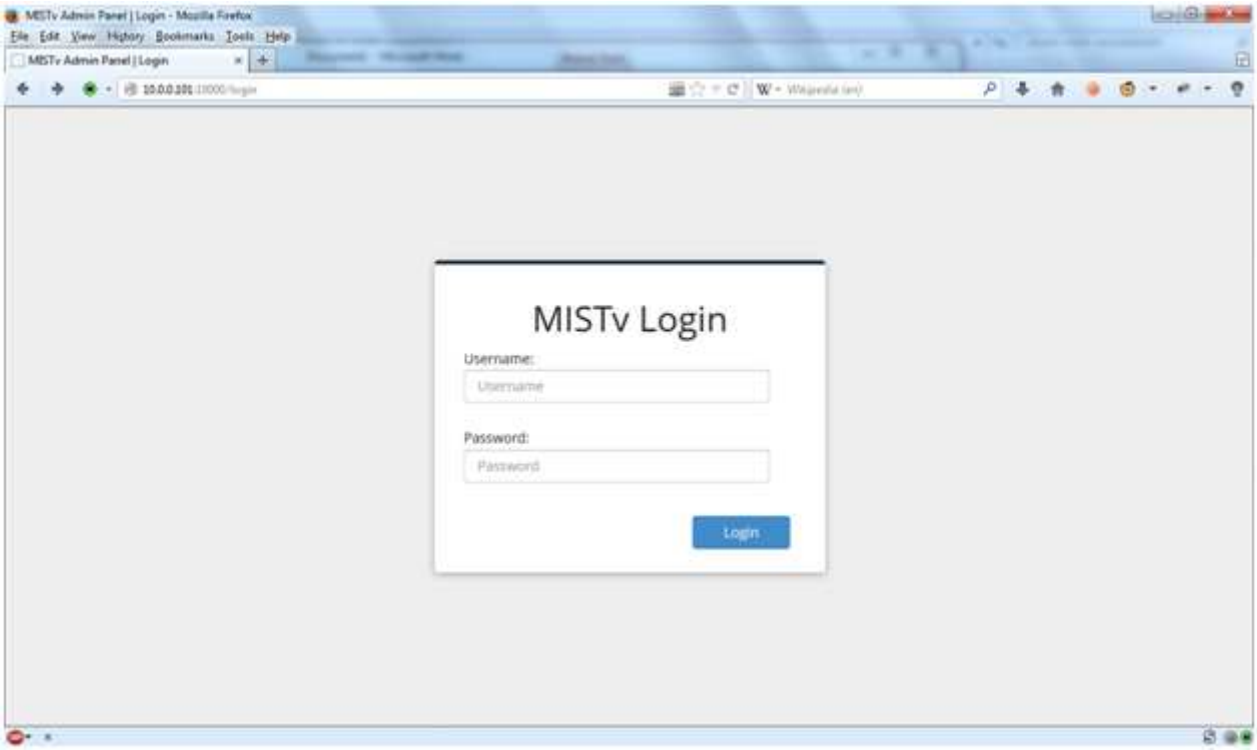

Figure 5.1: Screenshot of the log-in page interface

Authorization was implemented in the webserver as seen in Figure 5.1. This will require the user to input credentials before letting him inside the admin panel.

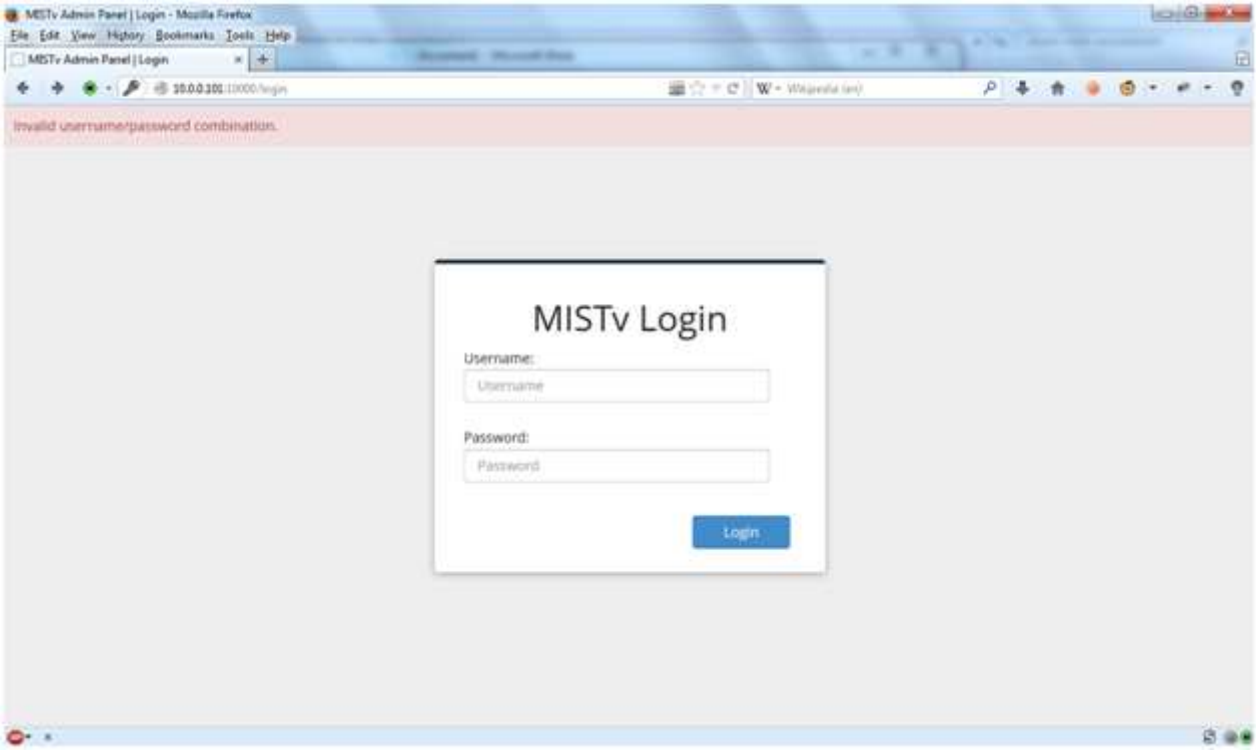

Figure 5.2: Log-in page interface - Prompt for invalid log-in credentials

Attempting to input an invalid username and password combination will redirect the user back to the login page as seen in Figure 5.2. Flash messages were implemented to show feedback to the users when logging in, uploading a file, or modifying a file. An example of a flash message can be seen in Figure 5.2 with the message "Invalid username/password combination."

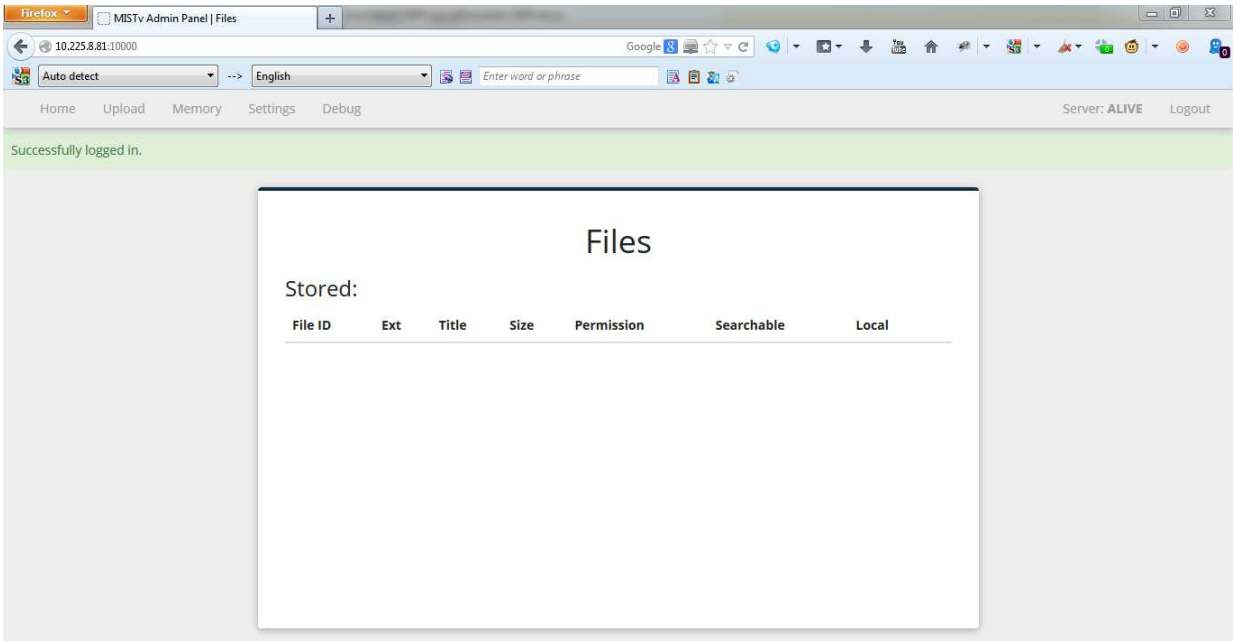

Figure 5.3: Screenshot of the home page upon successful log-on

After successfully logging in, the user is redirected to the file index page as seen in Figure 5.3. The list of files stored in the node of the user are shown here.

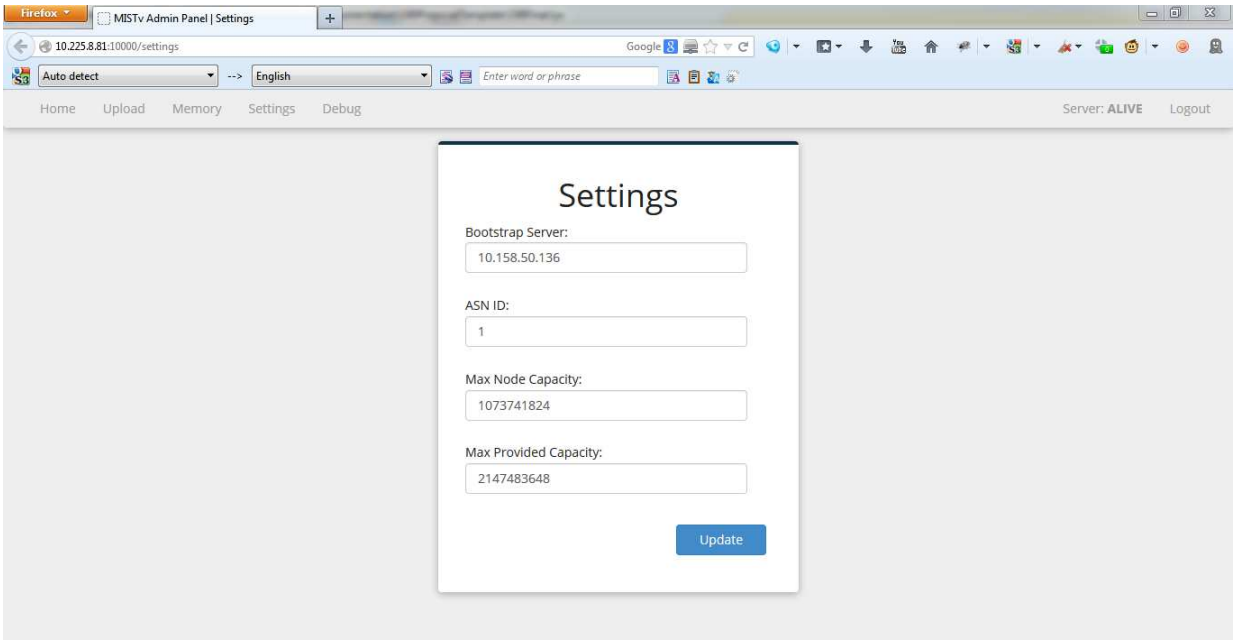

Figure 5.4: Screenshot of the settings page. Only the bootstrap server should be changeable. The rest of the parameters are for testing purposes.

Settings have also been implemented in the webserver so that it is now unnecessary to input these parameters when running the server as seen in Figure 5.4.

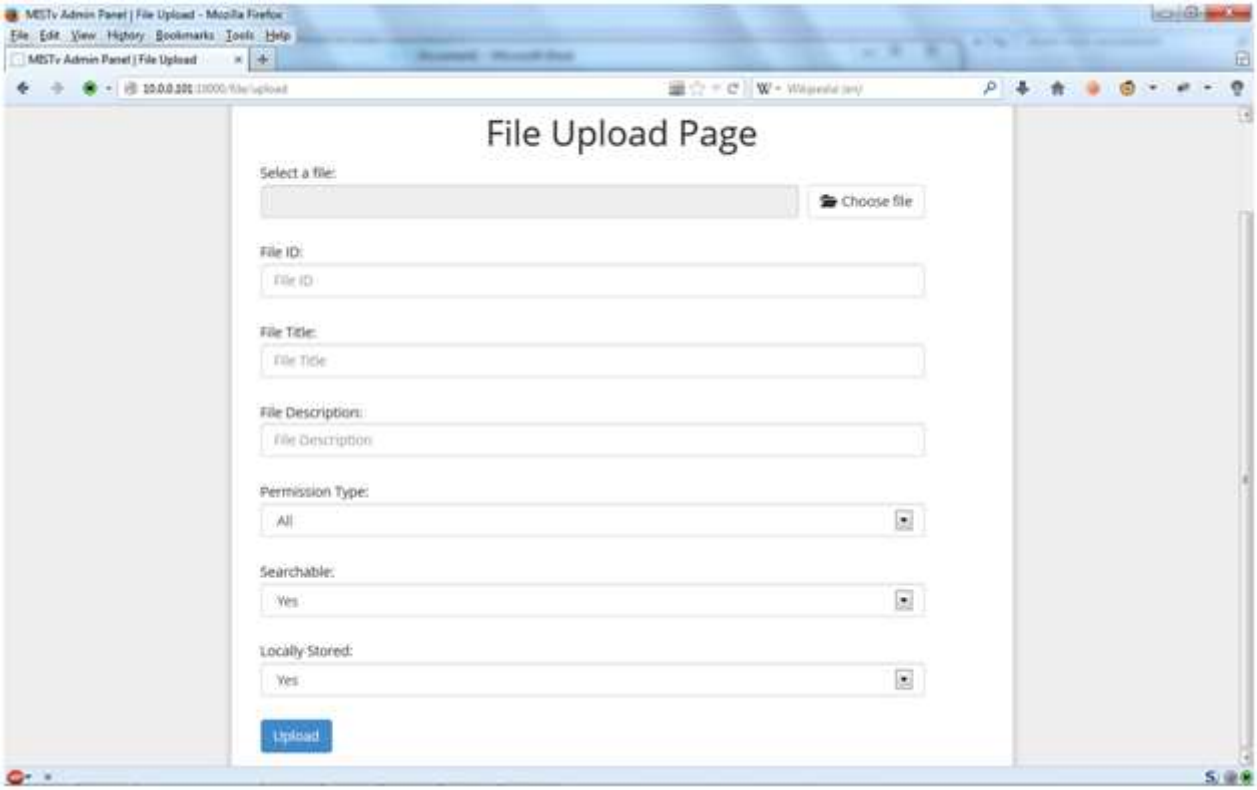

Figure 5.5: Screenshot of the file upload page

The file upload page has also been updated as seen in Figure 5.5. Users can also set the permission type and searchability of the file, as well as choose if they want to store their file on their own device or not.

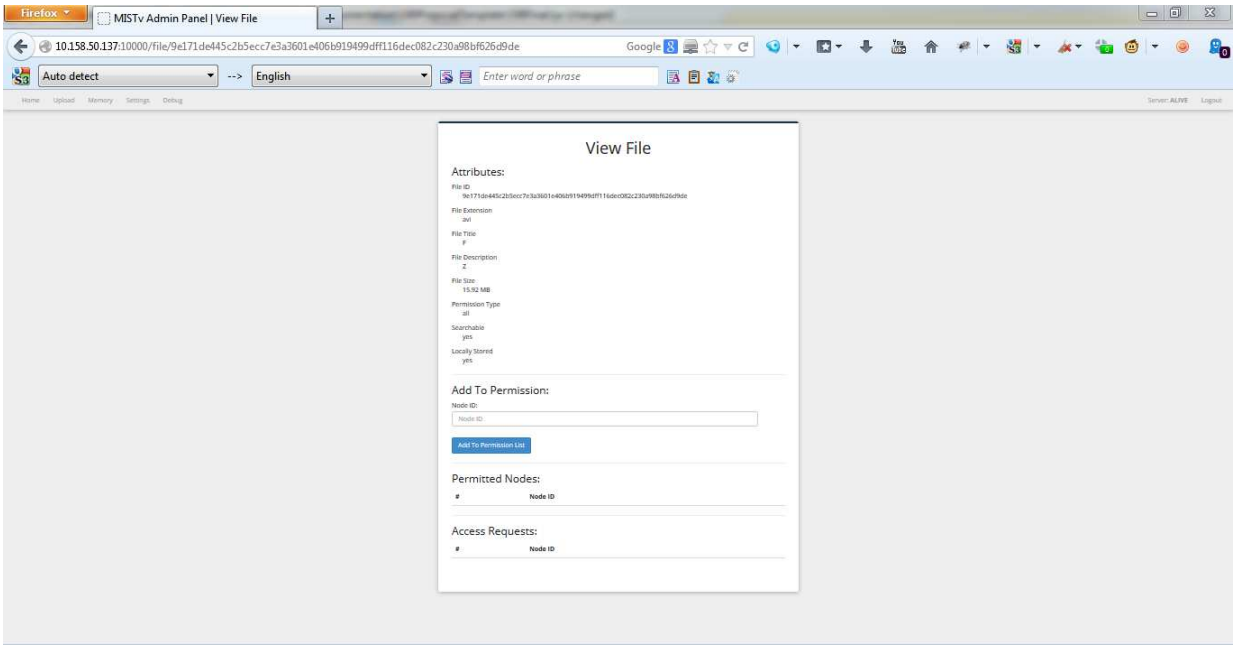

Figure 5.6: View file attributes page. Users can permit other users to access their files here as well.

The view file page is shown in Figure 5.6. Here we can see the attributes of the file, including the ones that the user has set. Users can also accept requests to access files in this page, as well as add users manually to allow them to access the file if the permission is set to "few".

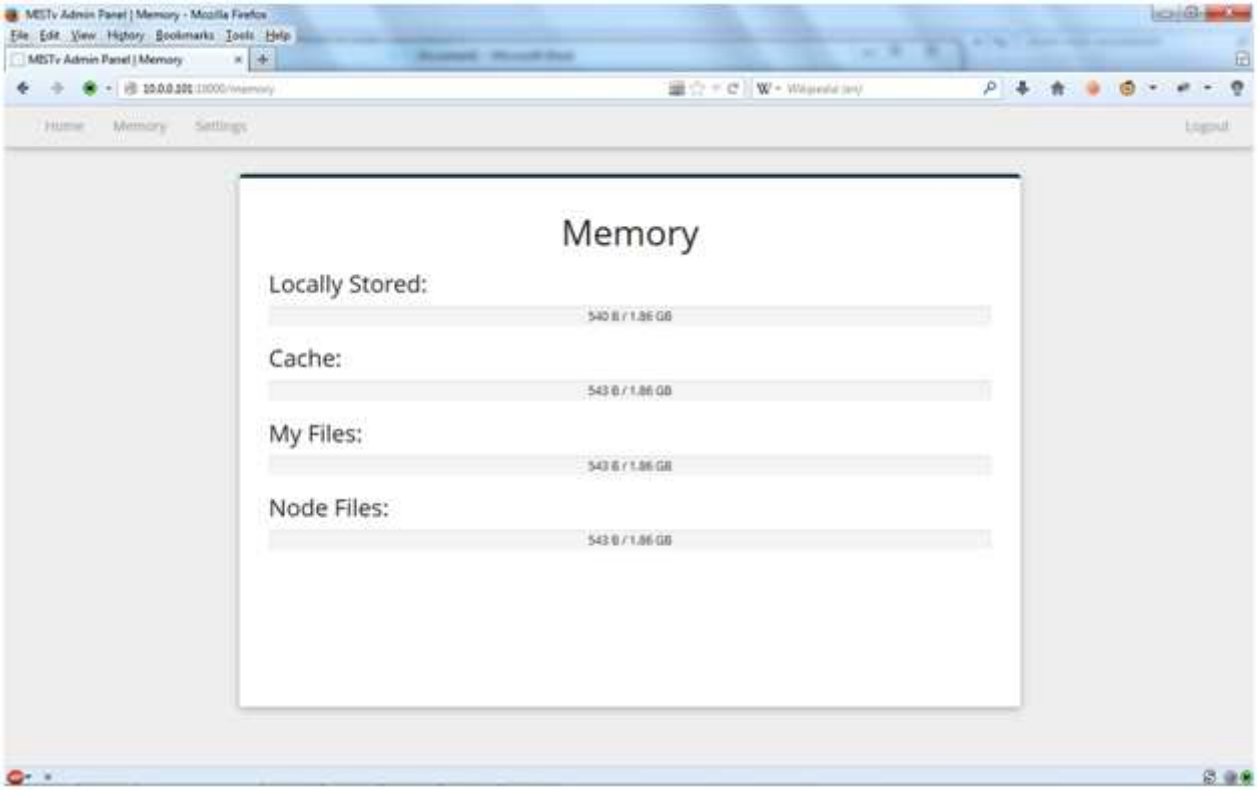

Figure 5.7: Screenshot of the memory statistics page

The memory page is presented in Figure 5.7. Users can view how much memory they have consumed in this page.
Our User Interface can use the Terminal Commands shown in Section 4.5 as seen in the screenshots above.

5.2 Video Streaming Quality

For video streaming tests, we decided to use two high-definition videos, one being an animated video with subtitles, and the other being a video file of an episode of a popular TV Series. The animated video shows artifacts from time to time, while the other video sometimes loses its audio for a while due to audio lag. There are also artifacts seen while playing the TV Series, however it is hardly noticeable even on a widescreen TV. The videos play steadily enough even if we are running them on a Raspberry Pi, making it a suitable entertainment system. Figures 5.8 and 5.9 below show screenshots of the animated video and the TV series playing on MISTv and

projected on a large widescreen TV.

]

Figure 5.8: Screenshot of the Animated Video Played Through MISTv

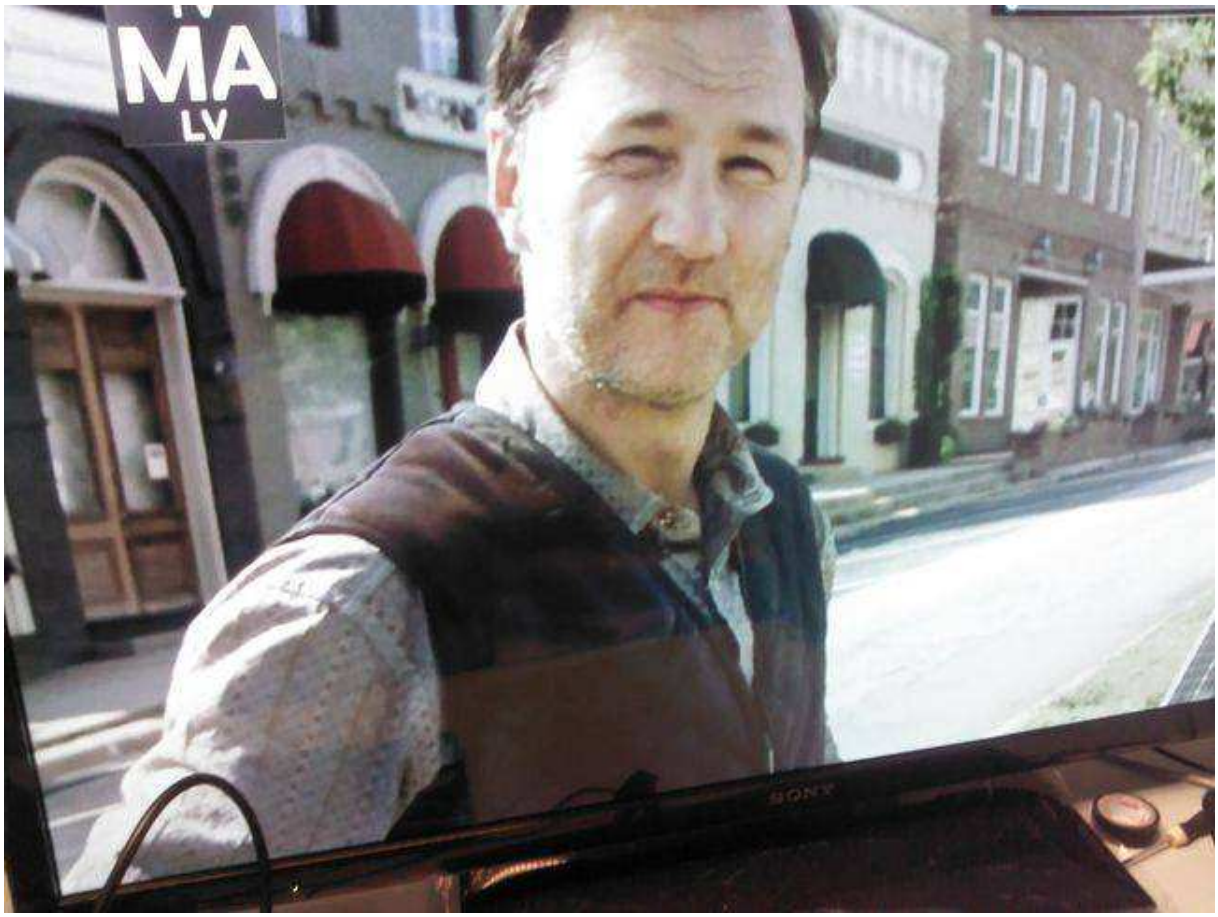

Figure 5.9: Screenshot of a TV Series Played Through MISTv

5.3 Download Speed Tests

The results of the download speed tests discussed in section 4.10 are shown in this section.

5.3.1 Chunk Size

Table 5.1: Relationship of Download Speed with Chunk Size. Increasing chunk size translates to decreasing average find time and better download speed until 4MB.

Chunk Size (MB)	Ave. Find Time (s)	Ave. Download Time (s)	Ave. Download Speed (kB/s)
1	766.9	10.0	1636.8
2	361.8	9.1	1793.7
4	102.1	8.2	1990.0
8	93.3	9.9	1658.1

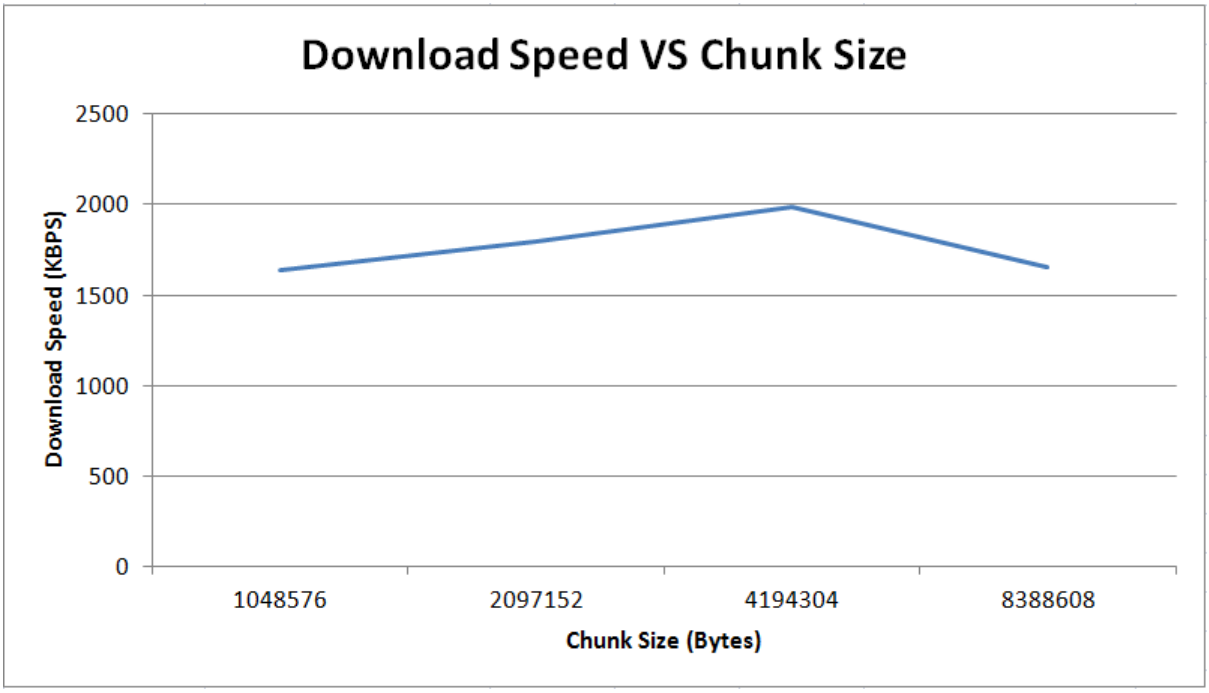

Figure 5.10: Logarithmic Graph of Chunk Size vs. Download Speed. Increasing chunk size translates to better download speed until 4MB.

Table 5.1 and Figure 5.10 above show the effect of the chunk size to the download speed. All other system variables were held constant. The security feature was also turned off while gathering data for this test because it had no influence on this experiment.

The results for chunk sizes 4MB and less is expected because as we decrease the chunk size, we at least double the time used by Kademlia's iterative function. However, when we tested the download speed with a chunk size of 8 MB, the find function took approximately 93 seconds,

Table 5.2: Chunk Size Test with a Larger File

Chunk Size (MB)	Ave. Find Time (s)	Ave. Download Time (s)	Ave. Download Speed (kB/s)
4	754.0	35.5	1963.0
8	441.6	35.7	1957.4

only a nine second reduction from the approximately 102 second finding time needed by the download with a chunk size of 4 MB. Also, the download speed with a chunk size of 8 MB is lower than thew download speed with a chunk size of 4 MB. The reason for the anomaly in the time taken by the iterative find function is that the number of chunks in the 4 MB case is 4 chunks for our 16 MB file, which makes a little difference to the number of chunks of the 8 MB scenario, which is 2 chunks. The anomaly of the speed drop of the 8 MB scenario was now caused by the inability of our code to get an entire 8 MB chunk in an instant.

Table 5.2 shows the results of a second round of testing with a 68 MB file. Here we see that the average find time of the 4 MB chunk size test is almost double the time needed by the 8 MB chunk size test due to the number of chunks resulting from these chunk sizes. We also see the same anomaly where the download speed of the 8 MB chunk size test became smaller than the download speed of the 4 MB chunk size test due to the same reason.

5.3.2 Mini Chunk Size

Table 5.3: Relationship of Download Speed with Mini Chunk Size. Increasing mini chunk size translates to better download speed.

Mini Chunk Size (kB)	Ave. Find Time (s)	Ave. Download Time (s)	Ave. Download Speed (kB/s)
32	34.7	18.0	903.7
64	35.6	12.0	1353.8
128	34.8	7.3	2223.8
256	35.2	6.5	2523.6
512	35.6	5.7	2871.1

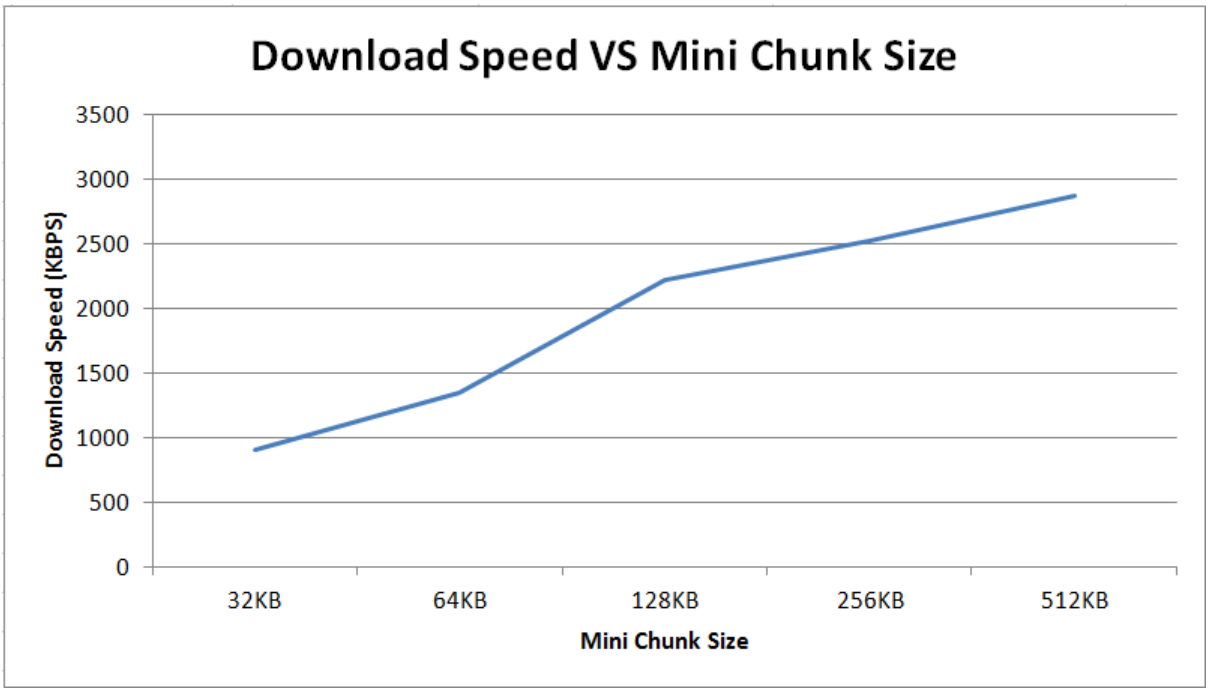

Figure 5.11: Logarithmic Graph of Mini Chunk Size vs. Download Speed. Increasing mini chunk size translates to better download speed.

Table 5.3 and Figure 5.11 above illustrate the effect of the mini chunk size to the download speed. All other system variables were held constant. The security feature was also turned off while gathering data for this test because it had no influence on this experiment. The iterative function also has no effect on the download performance of MISTv in relation to mini chunk size since it constantly used approximately 35 seconds per test size.

As we can see from the graph and table presented, every time we cut the chunk size in half, the download speed also decreases by at least 300kB. This is because smaller mini chunk sizes means that XBMC needs to query its own node server more often so that it can update the progress bar, which leads the program to do more tasks compared to when we use larger mini chunk sizes. Therefore, we must choose the largest mini chunk size for a granular progress bar for users to see, which is 512kB in this case.

5.3.3 Alpha and K

Table 5.4: Relationship of Download Speed with Alpha and K. Increasing Alpha slows down the average find time. Decreasing K reduces average find time at the cost of a lower find success rate.

Alpha	K	Ave. Find Time (s)	Ave. Download Time (s)	Ave. Download Speed (kB/s)
3	20	102.1	7.0	1990.0
6	12	92.0	7.1	2162.8
8	16	131.8	5.3	2112.2
10	20	172.5	4.1	2115.0

Table 5.4 above presents the relationship of alpha, k, the average iterative search time, the average download time, and the average download speed. The test values for the two variables were based on [26]. The download speeds are quite close to each other, so the download speed does not influence the decision of what the best alpha and k are. Instead, we can see that as we increase the value of alpha or k, we increase the average time used by the iterative find function we coded. So, it is better to choose values of alpha and k that yield relatively small average iterative find times. However, making the value of k too small results causes our system to fail in finding files over the network. When we set k = 10, the system failed to find the file we uploaded over the network. So, it is best to choose a relatively large value of k and a small alpha. Therefore, the best values for alpha and k in our experiment are 3 and 20, respectively.

5.3.4 Encryption

Table 5.5: Effect of File Encryption with Download Speed. Enabling encryption lowers download speed.

Encrypted?	Ave. Find Time (s)	Ave. Download Time (s)	Ave. Download Speed (kB/s)
True	168.5	8.6	1955.2
False	167.3	8.0	2042.6

Table 5.5 above shows us the difference in download speed if we encrypt files over the network or not. As we can see from the table, the download speed with file encryption is only about 100 kB/s slower compared to the download speed without file encryption. There is also practically no difference between the average find times between the two cases. However, we only have four chunks for this test case, meaning that each chunk adds approximately 0.15 seconds to the download time. This means that encrypting files and distributing them over the network does

Table 5.6: Download Speed with Localization. The experiment yields inconclusive results.

# of Local Nodes	Ave. Find Time (s)	Ave. Download Time (s)	Ave. Download Speed (s)
5	168.8	8.4	1939.0
10	170.6	8.2	1989.2
15	169.1	7.9	2072.7

not have much impact on the performance of the entire system if the number of chunks are few, making file encryption worth using for small files.

5.3.5 Localization

Table 5.6 above shows the relationship between the number of local nodes and the average download speed. In this test, the inter-pseudo ISP delay used was the one stated in section 4.8. The ASN's were randomly assigned to each of the nodes while ensuring that the number of nodes with the same ISP for testing purposes was fixed to 5, 10, and 15. Encryption was also turned off in this experiment, the chunk size was set to 4 MB, the mini chunk size was set to 256 kB, alpha was set to 3, and k was set to 20. Based on the results of the average download speed, it seems that as we increase the number of nodes, we also increase the download speed. This means that it is necessary to have a sufficient number of nodes in your locality that share the same file in order for localization to be beneficial to both the end users and the ISP's. If the chunk replication was not sufficient, we may have not been able to successfully download the file in the 5 local node case. However, the increase in the download speed and the variations in the average find time are quite small. The delays are also barely noticeable because there are only 4 chunks to download, thus minimizing the delay time. The problem with these measurements are that the download is fast in the local area network, there was no induced loss introduced in the network, and there were only 4 chunks to download. This makes the effect of localization seem insignificant in our testbed.

5.3.6 Cost of Secure Communication

All the download tests showed us that the iterative function takes up at least double the time it takes to download the chunks over the network. This is because of the secure communication system discussed in section 4.9.1. The iterative function always uses the secure communication system because we need to prevent attacks to the network and the users. Taking a look at client.py (see Appendix B.1), the client program of each peer in the network, and using a python script called 'line profiler', we saw that the iterative function used by both get_value() and get_nearest_nodes()

is called 100 times each in our test case (see Figure 5.12), and that the program spends almost 100% of the time running these slow functions that use approximately 1.5 seconds per iteration. Underneath this get_value() function, there is a function called protect_data() that takes 78% of the request time (see Figures 5.13 and 5.14). This means that protect_data() takes approximately 1.17 seconds to run per iteration, which is quite slow considering how many times it needs to be called. This is because the protect_data() function has a step that involves AES key generation that uses PBKDF2, which is a considerably slow algorithm when it is run on a normal Raspberry Pi based on our experiments. We even had to lower the number of PBKDF2 iterations from 5000 to 32, which is significantly below the recommended value of 1000 [54]. The cost of securing communication between the Raspberry Pi's is a great drop in speed and performance.

```
522
523         5          224     44.8      0.0    if function_type == ITERATIVE_NEAREST_NODES:
524                                                 nodes = get_nearest_nodes(my_node['server'], my_node['port'], key_id, CONST_ALPHA)
525                                               else: #value
526         5     11188900 2237780.0      3.5       return_message = get_value(my_node['server'], my_node['port'], key_id, CONST_ALPHA)
527         5          397     79.4      0.0       if return_message['found'] == "True":
528                                                   del return_message['found']
529                                                   nodes = get_nearest_nodes(my_node['server'], my_node['port'], key_id, CONST_ALPHA)
530                                                   my_values[my_node['node_id']] = return_message
531                                                   node_map[my_node['node_id']] = (my_node['server'], my_node['port'])
532         5          275     55.0      0.0       elif return_message['found'] == "False":
533         5          254     50.8      0.0         del return_message['found']
534         5          341     68.2      0.0         nodes = return_message
535
536         5          229     45.8      0.0    if len(nodes) == 0:
537                                                 return None
538
539         5          462     92.4      0.0    node_map.update(nodes)
540         5         1783    356.6      0.0    short_list_append(short_list, nodes)
541
542         5          276     55.2      0.0    def key_function(node_key):
543                                                 return int(key_id, 16) ^ int(node_key, 16)
544
545         5         7707   1541.4      0.0    alpha_contacts = heapq.nsmallest(CONST_ALPHA, short_list.keys(), key_function)
546         5          235     47.0      0.0    closest_node = alpha_contacts[0]
547
548         5          185     37.0      0.0    while 1:
549       110         6639     60.4      0.0      for contact in alpha_contacts:
550       100         4707     47.1      0.0        if function_type == ITERATIVE_NEAREST_NODES:
551                                                   nodes = get_nearest_nodes(node_map[contact][0], node_map[contact][1], key_id, CONST_K)
552                                                 else: #value
553       100    147303772 1473037.7     46.7          return_message = get_value(node_map[contact][0], node_map[contact][1], key_id, CONST_K)
554       100         8802     88.0      0.0          if return_message['found'] == "True":
555       100         7659     76.6      0.0            del return_message['found']
556       100    145193802 1451938.0     46.0             nodes = get_nearest_nodes(node_map[contact][0], node_map[contact][1], key_id, CONST_K)
557       100         8966     89.7      0.0            my_values[contact] = return_message
558                                                   elif return_message['found'] == "False":
559                                                     del return_message['found']
560                                                     nodes = return_message
561
562       100        12061    120.6      0.0        if len(nodes) != 0 or len(return_message) != 0:
563       100         7842     78.4      0.0          short_list[contact] = True
564       100     11415963 114159.6      3.6          bucket_request(contact, node_map[contact][0], node_map[contact][1])
565       100        30377    303.8      0.0          node_map.update(nodes)
566       100        48799    488.0      0.0          short_list_append(short_list, nodes)
567                                                 else:
568                                                   del short_list[contact]
```

Figure 5.12: Usage of get_value in client.py according to line profiler

```
Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
   474                                           @profile
   475                                           def request(server, port, key_id, n, action):
   476       205        46094    224.8      0.0    url = build_url(server, port, action)
   477
   478       205        16734     81.6      0.0    data = { 'key': key_id,
   479       205        10756     52.5      0.0             'n': n,
   480       205        11150     54.4      0.0             'peer_id': my_node['node_id'],
   481       205        17392     84.8      0.0             'peer_server': my_node['server'],
   482       205        10370     50.6      0.0             'peer_port': my_node['port'] }
   483       205     19700140  96098.2      6.6    receiver_id = get_node_id(server)
   484
   485       205    235378949 1148190.0     78.6     protect_data(receiver_id=receiver_id, data=data)
   486
   487       205         9540     46.5      0.0    try:
   488       205     43810256 213708.6     14.6      r = requests.get(url, data=data)
   489                                           except requests.exceptions.RequestException:
   490                                             return {}
   491
   492       205        22595    110.2      0.0    if r.status_code == requests.codes.ok:
   493       205       381744   1862.2      0.1      return r.json()
   494                                           else:
   495                                             return {}
```

Figure 5.13: Usage of protect_data in client.py according to line profiler

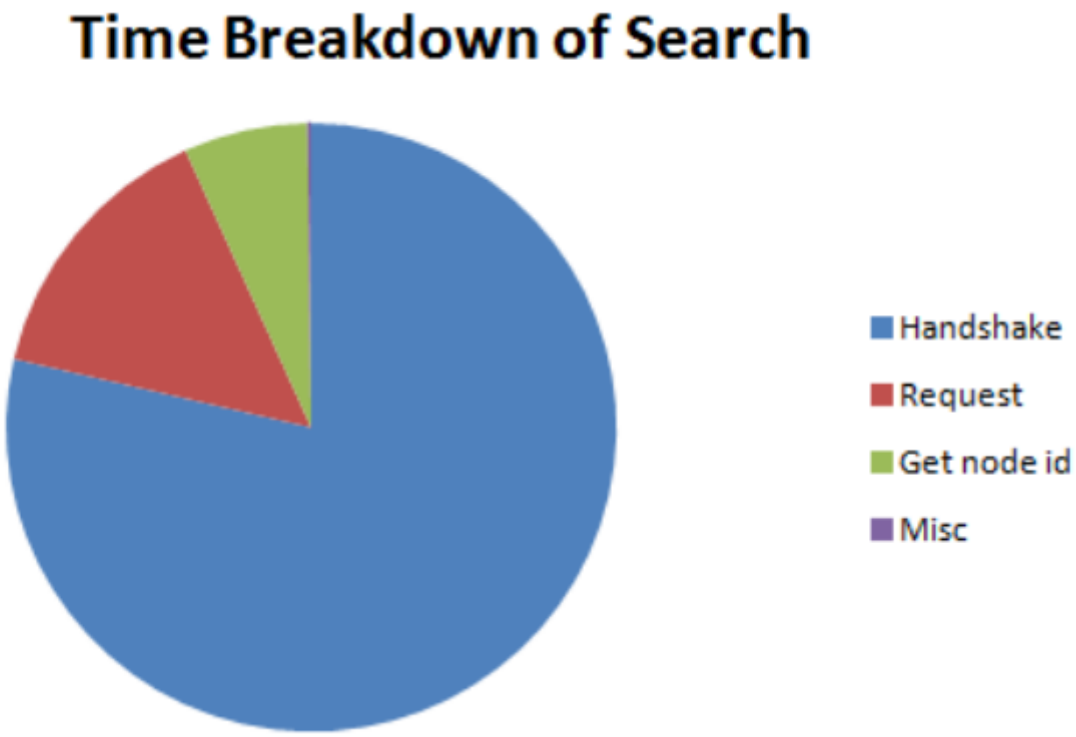

Figure 5.14: Visual representation of the usage of protect_data in client.py

Chapter 6

Conclusions and Recommendations

MISTv, a system that allows end users to seamlessly watch video on a television and store files over a locality-aware Kademlia-based DHT peer-to-peer local area network via a low-cost single-board computer was successfully developed in Python. Users are also able to interact and control their devices using any web browser of their preference. We were also able to determine the effects of the system parameters such as the chunk size, mini chunk size, alpha, k, and encryption on the download speed and average node or file search of the system. The system is also able to encrypt files with a working permission scheme while maintaining the peer-to-peer nature of the system. However, the permission scheme is flawed because the private key being shared is unencrypted, which means that eavesdroppers can steal these private keys quite easily. Encrypting these keys every transaction will slow down transactions for encrypted files. The private key for encrypted files also has to be requested from the owner of the file, which affects the peer-to-peer nature of the system. So, we recommend that a more advanced security system must be used to securely share files in a peer-to-peer fashion.

Experiments on the performance of MISTv show that the download speed of a single client connected to the network is in the megabytes range, but the time taken by the iterative function, which is at least double the time taken by the download, is too large due to the secure communication system. We recommend the use of hardware acceleration, the installation of extra hardware for cryptography, or choosing a device with more processing power so that the iterative function does not drag down the performance of the system.

Our code is also multi-threaded, which consumes a good amount of resources. So, we recommend to modify our code such that it follows the asynchronous programming model, which

uses resources quite efficiently.

Localization tests showed inconclusive results. The tests were done in an almost ideal environment of minimal loss, small downloads, and in a local area network. So, we recommend that for future tests that you measure the performance of the system with localization in a lossy network, with a setup that has nodes on separate networks, and with larger test files.

Bibliography

[1] I. Pita, "A formal specification of the kademlia distributed hash table," in *Internet: http://maude.sip.ucm.es/kademlia/files/pita_kademlia.pdf, [Sept. 1, 2013].*

[2] "Kademlia," in *Internet: http://en.wikipedia.org/wiki/Kademlia, [Sept. 1, 2013].*

[3] "Video content and syndication: Long-form content on the rise," in *Internet: https://projects.pbs.org/confluence/download/attachments/16746313/eMarketer_Video_Content_and_Streaming_Syndication-Long-Form_Content_on_the_Rise_0710.pdf, [Aug. 13, 2013].*

[4] "Youtube press statistics," in *Internet: http://www.youtube.com/yt/press/statistics.html, [Aug. 13, 2013].*

[5] "Sandvine global internet phenomena report," in *Internet: http://www.sandvine.com/downloads/documents/Phenomena_1H_2012/Sandvine_Global_Internet_Phenomena_Report_1H_2012.pdf, 2012 [Aug. 13, 2013].*

[6] "Gartner survey shows data growth as the largest data center infrastructure challenge," in *Internet: http://www.gartner.com/newsroom/id/1460213, 2010 [Aug. 13, 2013].*

[7] V. Barret, "Dropbox: the inside story of tech's hottest start-up," in *Internet: http://www.forbes.com/sites/victoriabarret/2011/10/18/dropbox-the-inside-story-of-techs-hottest-startup/, Oct. 2011 [Aug. 13, 2013].*

[8] C. Arthur, "Technology firms to spend $150bn on building new data centres," in *Internet: http://www.theguardian.com/business/2013/aug/23/spending-on-data-centres-reaches-150-billion-dollars, Aug. 23, 2013 [Sep. 30, 2013].*

[9] C. Preimesberger, "Unplanned it downtime can cost 5k per minute," in *Internet: http://www.eweek.com/c/a/IT-Infrastructure/Unplanned-IT-Downtime-Can-Cost-5K-Per-Minute-Report-549007/, May 2011 [Aug. 13, 2013].*

[10] C. Harris, "It downtime costs $26.5 billion in lost revenue," in *Internet: https://www.informationweek.com/storage/disaster-recovery/it-downtime-costs-265-billionlost-re/229625441, May 2011 [Aug. 13, 2013].*

[11] S. Higginbotham, "Peering pressure: the secret battle to control the future of the internet," in *http://gigaom.com/2013/06/19/peering-pressure-the-secret-battle-to-control-the-future-of-the-internet/, Jun. 2013 [Aug. 13, 2013].*

[12] D. R. Choffnes and F. E. Bustamante, "Taming the torrent: A practical approach to reducing cross-isp traffic in peer-to-peer systems," in *ACM SIGCOMM*, 2008.

[13] R. F. C. Quevedo, G.P.L.; Ocampo, "Evaluating the effects of peer localizlocal on a bittorrent-based p2p video-on-demand network," in *TENCON 2012 - 2012 IEEE Region 10 Conference*, pp. 1–5, November 2012.

[14] J. P. J. A. K. T. A. M. Piatek, H. V. Madhyastha, "Pitfalls for isp-friendly p2p design," in *Hotnets*, 2009.

[15] M. S. M. Varvello, "Traffic localization for dht-based bittorrent networks," in *Networking'11*, (Valencia, Spain), May 2011.

[16] M. S. M. Varvello, "Dht-based traffic localization in the wild," in *2013 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 103,108, April 2013.

[17] Z. X. E. T. X. D. L. Guo, S. Chen and X. Zhang, "Measurements, analysis, and modeling of bittorrent-like systems," in *Proc. of the 5th ACM SIGCOMM conference on Internet measurement*, (CA, USA), pp. 35–48, 2005.

[18] H.-C. Hsiao and C.-T. King, "Modeling and evaluating peer-to-peer storage architectures," in *Parallel and Distributed Processing Symposium Proceedings International, IPDPS 2002, Abstracts and CD-ROM*, vol. vol., no.„ p. 6, 2001.

[19] R. Hasan, Z. Anwar, W. Yurcik, L. Brumbaugh, and R. Campbell, "A survey of peer-topeer storage techniques for distributed file systems," *Information Technology: Coding and Computing, 2005. ITCC 2005. International Conference on*, vol. 2, pp. 205,213, 2005.

[20] B. Cohen, "Incentives build robustness in bittorrent," in *Workshop on Economics of Peer-to-Peer Systems*, 2003.

[21] I. C. et. al., "Freenet: A distributed anonymous information storage and retrieval system," in *Workshop on Design Issues in Anonymity and Unobservability*, pp. 46–66, 2000.

[22] I. et. al., "Kelips: Building an efficient and stable p2p dht through increased memory and background overhead," in *Proceedings of the 2nd International Workshop on Peer-to-Peer Systems*, 2003.

[23] A. Rowstron and P. Druschel, "Storage management and caching in past, a largescale, persistent peer-to-peer storage utility," in *ACM SOSP*, 2001.

[24] P. Maymounkov and D. Mazieres, "Kademlia: A peer-to-peer information system based on the xor metric," in *Internet: http://www.cs.rice.edu/Conferences/IPTPS02/109.pdf*, 2002.

[25] N. Fedotova, S. Fanti, and L. Veltri, "Kademlia for data storage and retrieval in enterprise networks," *International Conference on Collaborative Computing: Networking, Applications and Worksharing, 2007. CollaborateCom 2007*, vol. vol., no.„ pp. 382–386, 2007.

[26] I. Baumgart and S. Mies, "S/kademlia: A practicable approach towards secure key-based routing," in *Parallel and Distributed Systems, 2007 International Conference on , vol.2, no*, pp. 2–3, 2007.

[27] M. Kohnen, "Analysis and optimization of routing trust values in a kademlia-based distributed hash table in a malicious environment," *Future Internet Communications (BCFIC), 2012 2nd Baltic Congress on*, vol. vol., no.„ pp. 252–259, 2012.

[28] "Bittorrent sync," in *Internet: http://labs.bittorrent.com/experiments/sync/technology.html, [Aug. 11, 2013].*

[29] "Raspberry pi faqs," in *Internet: http://www.raspberrypi.org/faqs, [Sept. 13, 2013].*

[30] "Raspberry pi downloads," in *Internet: http://www.raspberrypi.org/downloads, [Sept. 13, 2013].*

[31] "Raspberry pi raspbian forums," in *Internet: http://www.raspberrypi.org/phpBB3/viewforum.php?f=66&sid=144bd6a8e43788527a62d8c33fad1986, [Sept. 13, 2013].*

[32] "Xbmc media center," in *Internet: http://xbmc.org/, [Dec. 14, 2013].*

[33] "Plex," in *Internet: http://www.plexapp.com/, [Dec. 14, 2013].*

[34] "Openelec," in *Internet: http://openelec.tv/, [Dec. 14, 2013].*

[35] "Raspbmc," in *Internet: http://www.raspbmc.com/, [Dec. 14, 2013].*

[36] "Xbian," in *Internet: http://www.xbian.org, [Dec. 14, 2013].*

[37] "Rasplex," in *Internet: http://rasplex.com/, [Dec. 14, 2013].*

[38] T. Klosowski, "Raspberry pi xbmc solutions compared: Raspbmc vs openelec vs xbian," in *Internet: http://lifehacker.com/raspberry-pi-xbmc-solutions-compared-raspbmc-vs-openel-1394239600, Sept. 26, 2013 [Dec. 14, 2013].*

[39] "Cec (consumer electronics control) over hdmi," in *Internet: http://elinux.org/CEC_%28Consumer_Electronics_Control%29_over_HDMI, [Sept. 16, 2013].*

[40] xbianonpi, "qemu-env," in *Internet: https://github.com/xbianonpi/qemu-env, Apr, 29, 2013 [Dec. 13, 2013].*

[41] "Qemu - emulating raspberry pi the easy way (linux or windows!)," in *Internet: http://xecdesign.com/qemu-emulating-raspberry-pi-the-easy-way/, Apr. 15, 2012 [Dec. 13, 2013].*

[42] F. Aucamp, "Entangled," in *Internet: http://entangled.sourceforge.com, 2008 [Sept. 1, 2013].*

[43] A. Swartz, "Khashmir," in *Internet: http://khashmir.sourceforge.net/, [Sept. 1, 2013].*

[44] I. Zafuta, "Pydht," in *Internet: https://github.com/isaaczafuta/pydht, Jul. 29, 2013 [Dec. 15, 2013].*

[45] "Nightmare," in *Internet: https://github.com/volker48/Nightmare, 2011 [Dec. 15, 2013].*

[46] "Requests: Http for humans," in *Internet: http://requests.readthedocs.org/en/latest/, [Dec. 15, 2013].*

[47] "Bottle: Python web framework," in *Internet: http://bottlepy.org/docs/dev/, [Dec. 14, 2013].*

[48] "Cherrypy," in *Internet: http://www.cherrypy.org/, [Dec. 14, 2013].*

[49] "How fast is cherrypy?," in *Internet: http://cherrypy.readthedocs.org/en/latest/appendix/cherrypyspeed.html, 2013 [Dec. 15, 2013].*

[50] G. Harrison, "10 things you should know about nosql databases," in *Internet: http://www.techrepublic.com/blog/10-things/10-things-you-should-know-about-nosql-databases/, Aug. 26, 2010 [Dec. 15, 2013].*

[51] A. Negron, "Gentle introduction to mongodb using pymongo," in *Internet: http://altons.github.io/python/2013/01/21/gentle-introduction-to-mongodb-using-pymongo/, [Dec. 15, 2013].*

[52] K. Isom, "A working introduction to crypto with pycrypto," in *Internet: http://kyleisom.net/downloads/crypto_intro.pdf, Feb. 9, 2013 [Dec. 13, 2013].*

[53] D. Litzenberger, "Pycrypto - the python cryptography toolkit," in *Internet: https://www.dlitz.net/software/pycrypto/, [Dec. 13, 2013].*

[54] "Pycrypto api documentation," in *Internet: https://www.dlitz.net/software/pycrypto/api/current/, [Dec. 13, 2013].*

[55] L. Luce, "Python and cryptography with pycrypto," in *Internet: http://www.laurentluce.com/posts/python-and-cryptography-with-pycrypto/, Apr. 22, 2011 [Dec. 13, 2013].*

Appendix A

Original Kademlia Algorithm

The following information was taken from the first paper to present the original Kademlia algorithm [24].

All nodes and files have unique fixed-length IDs which belong to the same keyspace. Files are stored to nodes with IDs which are closest to the ID of the file. The distance between ID's are measured by the XOR function.

Each node keeps a list for each bit of the node ID (i.e. if the node ID has 160 bits, it keeps 160 lists). Each list (also known as K-buckets) corresponds to a specific range in an ID space with no overlap. All node IDs in the entries in a particular list have a particular common prefix (see Figures A.1 and A.2). The list contains entries that specifies data used to locate a particular node (e.g. node ID - IP address, port number). A list can have up to a specified maximum number k of entries. This list of lists is the node's own directory.

The K-buckets are sorted based on most recent communication. The nodes which have communicated recently are pushed up to the tail of the list regardless of which node initiated the communication. Every time a node A receives a message from another node C, A updates the K-bucket C belongs to. If node C already exists in the bucket it is moved to the tail. If it is a new contact however and the bucket is not full it is added to the tail. However if the bucket is full, the node B which was least recently communicated with will be sent a PING message (see Table A.1); if B is dead, then the node C replaces B, otherwise C is dropped.

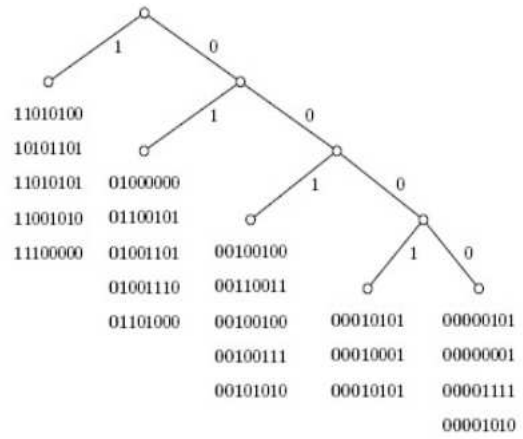

Figure A.1: List entries for node 00000000 of a 2^8 network. Image taken from [1]

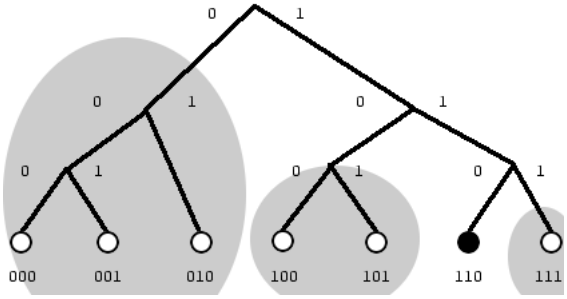

Figure A.2: List entries for node 110 of a 2^3 network. Image taken from [2]

Kademlia has four basic protocol messages: PING, STORE, FIND_CLOSEST_NODES, and FIND_VALUE (see Table A.1). These messages are the building blocks for all operations in the network, which are iterative in nature, are namely (1) node lookup, (2) store files, (3) searching for and getting a particular file, and (4) joining the network for a particular node, which will be discussed in the next few paragraphs.

Table A.1: Protocol Messages

Protocol Name	Function
PING	verifies if a node is online
STORE	commands a node to store the file ID and the corresponding file data to a particular node
FIND_CLOSEST_NODES	given a target ID, the receiving node returns a list of contact information of the K nodes it knows from its own lists which are closest to the target ID
FIND_VALUE	same as FIND_CLOSEST_NODES except if the recipient contains the file data corresponding to the target ID, it returns this instead of a list of contacts.

The node lookup operation is achieved by following the algorithm as follows. First, FIND_CLOSEST_NODES in its own lists. Second, FIND_CLOSEST_NODES in the K nodes as returned in the previous step, if the node contacted does not reply, then assume it is dead and take it out from the list prospect of nodes. Third, get the K closest nodes from what was returned in step 2. Lastly, repeat step 2 until the nodes returned at not closer than the nodes returned in the previous result, these nodes are the closest to the desired key.

In searching for a particular file, the same process is used as in the node lookup operation except that instead of using FIND_CLOSEST_NODES, FIND_VALUE is used. In the case that a file is received instead of a list of node contact information then the search is finished and the file is returned. To store a file, a STORE message is sent to all the nodes resulting from the node lookup operation. Finally, a node can join the network in the condition that it has a uniquely generated node ID and knows some the contact information of some node C in the network. It inserts node C in its own bucket and does a node lookup operation for its own ID (self-lookup). This self-lookup will populate the other nodes with its contact information. It will lookup a random ID within the range of the IDs in its lists to populate its own directory.

Appendix B

Source Code

B.1 Snippets of client.py

```
432  ##########################################################
433  # SECURITY
434  ##########################################################
435  def protect_data(receiver_id, data):
436    IV = EncryptFunctions.generateIV()
437    AESKey = EncryptFunctions.generateAESKey(iterations=32)
438    randomKey = os.urandom(32)
439
440    encryptedRandomKey = EncryptFunctions.encryptAES(data=randomKey, key=AESKey, iv=IV)
441
442    try:
443      rawRcvPubKey = bootstrap_key(receiver_id)
444    except ValueError:
445      print "Error: no key could be found for node_id %s" % (receiver_id)
446      return False
447
448    rcvPubKey = EncryptFunctions.createRSAFromStr(rawRcvPubKey)
449    encryptedAESKey = EncryptFunctions.encryptRSA(rcvPubKey=rcvPubKey, data=AESKey)
450
451    rawPubKey = EncryptFunctions.loadRawKey(PUBLIC_KEY_PATH)
452
453    prvKey = EncryptFunctions.loadKey(PRIVATE_KEY_PATH)
454    signature = EncryptFunctions.signFunction(prvKey=prvKey, data=randomKey)
455
456    data['security_iv'] = IV
457    data['security_encryptedRandomKey'] = encryptedRandomKey
458    data['security_encryptedAESKey'] = encryptedAESKey
459    data['security_publicKey'] = rawPubKey
460    data['security_signature'] = signature
461
462    return True
```

Figure B.1: Snippet 1 of client.py

```
464  def protect_file(key, path):
465      EncryptFile.encryptFile(key=key, inFilename=path)
466
467  def create_file_key(file_id):
468      AESKey = EncryptFunctions.generateAESKey(iterations=32)
469      return file_keys.create(file_id=file_id, key=AESKey)
470
471  ############################################################
472  # ITERATIVE FUNCTIONS
473  ############################################################
474  def request(server, port, key_id, n, action):
475      url = build_url(server, port, action)
476
477      data = { 'key': key_id,
478               'n': n,
479               'peer_id': my_node['node_id'],
480               'peer_server': my_node['server'],
481               'peer_port': my_node['port'] }
482      receiver_id = get_node_id(server)
483
484      protect_data(receiver_id=receiver_id, data=data)
485
486      try:
487          r = requests.get(url, data=data)
488      except requests.exceptions.RequestException:
489          return {}
490
491      if r.status_code == requests.codes.ok:
492          return r.json()
493      else:
494          return {}
```

Figure B.2: Snippet 2 of client.py

```
496 def get_nearest_nodes(server, port, key_id, n):
497     nodes = request(server, port, key_id, n, 'nearest-nodes')
498 
499     return nodes
500 
501 def get_value(server, port, key_id, n):
502     return request(server, port, key_id, n, 'value')
503 
504 def short_list_append(short_list, nodes):
505     for node_key in nodes.keys():
506         if node_key not in short_list:
507             short_list[node_key] = False
508 
509 def iterative_find_nodes(key_id):
510     return iterative_function(key_id, ITERATIVE_NEAREST_NODES)
511 
512 def iterative_find_value(key_id):
513     return iterative_function(key_id, ITERATIVE_VALUE)
514 
515 def iterative_function(key_id, function_type):
516     my_values = {}
517     closest_node = None
518     node_map = {}
519     short_list = {}
520     return_message = {}
```

Figure B.3: Snippet 3 of client.py

```
515 def iterative_function(key_id, function_type):
516     my_values = {}
517     closest_node = None
518     node_map = {}
519     short_list = {}
520     return_message = {}
521 
522     if function_type == ITERATIVE_NEAREST_NODES:
523         nodes = get_nearest_nodes(my_node['server'], my_node['port'], key_id, CONST_ALPHA)
524     else: #value
525         return_message = get_value(my_node['server'], my_node['port'], key_id, CONST_ALPHA)
526         if return_message['found'] == "True":
527             del return_message['found']
528             nodes = get_nearest_nodes(my_node['server'], my_node['port'], key_id, CONST_ALPHA)
529             my_values[my_node['node_id']] = return_message
530             node_map[my_node['node_id']] = (my_node['server'], my_node['port'])
531         elif return_message['found'] == "False":
532             del return_message['found']
533             nodes = return_message
534 
535     if len(nodes) == 0:
536         return None
537 
538     node_map.update(nodes)
539     short_list_append(short_list, nodes)
```

Figure B.4: Snippet 4 of client.py

```
541  def key_function(node_key):
542    return int(key_id, 16) ^ int(node_key, 16)
543
544  alpha_contacts = heapq.nsmallest(CONST_ALPHA, short_list.keys(), key_function)
545  closest_node = alpha_contacts[0]
546
547  while 1:
548    for contact in alpha_contacts:
549      if function_type == ITERATIVE_NEAREST_NODES:
550        nodes = get_nearest_nodes(node_map[contact][0], node_map[contact][1], key_id, CONST_K)
551      else: #value
552        return_message = get_value(node_map[contact][0], node_map[contact][1], key_id, CONST_K)
553        if return_message['found'] == "True":
554          del return_message['found']
555          nodes = get_nearest_nodes(node_map[contact][0], node_map[contact][1], key_id, CONST_K)
556          my_values[contact] = return_message
557        elif return_message['found'] == "False":
558          del return_message['found']
559          nodes = return_message
560
561      if len(nodes) != 0 or len(return_message) != 0:
562        short_list[contact] = True
563        bucket_request(contact, node_map[contact][0], node_map[contact][1])
564        node_map.update(nodes)
565        short_list_append(short_list, nodes)
566      else:
567        del short_list[contact]
568
569    closest_nodes_list = heapq.nsmallest(CONST_K, short_list.keys(), key_function)
570    shorter_list = {}
571
572    for key in closest_nodes_list:
573      shorter_list[key] = short_list[key]
```

Figure B.5: Snippet 5 of client.py

```
574
575         short_list = shorter_list
576
577         if False not in shorter_list.values():
578             break
579
580         uncalled_list = {k: v for k, v in short_list.iteritems() if not v}
581
582         if closest_nodes_list[0] != closest_node:
583             closest_node = closest_nodes_list[0]
584             alpha_contacts = heapq.nsmallest(CONST_ALPHA, uncalled_list.keys(), key_function)
585         else:
586             alpha_contacts = uncalled_list
587
588     for key in short_list.keys():
589         short_list[key] = node_map[key]
590
591     if function_type == ITERATIVE_NEAREST_NODES:
592         return short_list
593     else:
594         return my_values
```

Figure B.6: Snippet 6 of client.py

```
869 def join_protocol_open(data):
870     print "JOIN PROTOCOL OPEN"
871 
872     master_file_id = data['file_id']
873     chunk_size = int(data['chunk_size'])
874 
875     # For printing the statistics
876     protocol_start_time = time.time()
877     find_total_time = 0
878     download_total_time = 0
879 
880     # For upload progress
881     file_size = int(data['file_size'])
882     current_size = 0
883 
884     # Attempt to get the key if it's needed
885     if my_node['node_id'] == data['node_owner_id']:
886         key = file_keys.get_key(file_id=master_file_id)
887     elif data['permission_type'] == "few":
888         result = dht_request_access(data['node_owner_server'], data['file_id'])
889 
890         if result == "REJECTED":
891             upload_progress(file_id=master_file_id, has_key="Rejected")
892             print "CANT ACCESS"
893             return
894         else:
895             print "HAS ACCESS"
896             upload_progress(file_id=master_file_id, has_key="No")
897             key = base64.b64decode(result)
898     elif data['permission_type'] == "me":
899         upload_progress(file_id=master_file_id, has_key="Rejected")
900         print "CANT ACCESS"
901         return
```

Figure B.7: Snippet 7 of client.py

```
902   else:
903     print "HAS ACCESS"
904     upload_progress(file_id=master_file_id, has_key="No")
905     key = None
906
907   for index in range(1, int(data['number_of_chunks']) + 1):
908     find_start_time = time.time()
909
910     value = iterative_find_value(data[str(index)])
911
912     find_end_time = time.time()
913     find_total_time += find_end_time - find_start_time
914
915     if value == {}:
916       print "Could not find chunk #%d." % (data[str(index)])
917       print "JOIN PROTOCOL OPEN aborted."
918       return
919
920     # Localization
921     has_same_asn = False
922     for v in value.values():
923       asn = v['in'][FIND_VALUE_ASN]
924
925       if my_node['asn'] == int(asn):
926         has_same_asn = True
927         break
928
929     for v in value.values():
930       encrypted = v['encrypted']
931       server = v['in'][FIND_VALUE_SERVER]
932       port = v['in'][FIND_VALUE_PORT]
933       file_id = v['file_id']
```

Figure B.8: Snippet 8 of client.py

```
935         download_start_time = time.time()
936
937         #Localization
938         if LOCALIZATION and has_same_asn:
939           if my_node['asn'] != int(asn):
940             continue
941
942         if my_node['asn'] != asn:
943           path = "%s-%s" % (my_node['asn'], asn)
944           delay = asn_delays.get(path, 500)
945           print "Delay for path %s is %dms" % (path, delay)
946           time.sleep(delay)
947
948           path = "%s-%s" % (asn, my_node['asn'])
949           delay = asn_delays.get(path, 500)
950           print "Delay for path %s is %dms" % (path, delay)
951           time.sleep(delay)
952
953         #FIX ME: if fail download, try the next one, if successful, break
954         current_size = download_chunk(server=server, port=port, master_file_id=master_file_id,
955                                       file_id=file_id, file_size=file_size, current_size=current_size,
956                                       encrypted=encrypted, key=key)
957
958         download_end_time = time.time()
959         download_total_time += download_end_time - download_start_time
960
961         v['storage_type'] = "cache"
962         new_file_data(my_node['server'], my_node['port'], v)
963
964         break
```

Figure B.9: Snippet 9 of client.py

```
966     # For printing the download speed
967     protocol_end_time = time.time()
968     protocol_total_time = protocol_end_time - protocol_start_time
969     find_breakdown = find_total_time/protocol_total_time*100.0
970     download_breakdown = download_total_time/protocol_total_time*100.0
971     misc_breakdown = 100 - find_breakdown - download_breakdown
972     file_size_kb = file_size/1024.0
973     file_size_mb = file_size/1024.0/1024.0
974     average_speed_kb = file_size/download_total_time/1024.0
975     average_speed_mb = file_size/download_total_time/1024.0/1024.0
976     encrypted = (data['permission_type'] != "all")
977
978     print "File downloaded successfully"
979     print "Encrypted: %s" % (encrypted)
980     print "Total Seconds Elapsed: %.2f" % (protocol_total_time)
981     print "Total Minutes Elapsed: %.2f" % (protocol_total_time/60)
982     print "=== Time Breakdown ==="
983     print "Find %%: %.2f%%" % (find_breakdown)
984     print "Download %%: %.2f%%" % (download_breakdown)
985     print "Misc %%: %.2f%%" % (misc_breakdown)
986     print "=== Iterative Find Value Statistics ==="
987     print "Seconds Elapsed: %.2f" % (find_total_time)
988     print "Minutes Elapsed: %.2f" % (find_total_time/60)
989     print "=== Download Statistics ==="
990     print "Seconds Elapsed: %.2f" % (download_total_time)
991     print "Minutes Elapsed: %.2f" % (download_total_time/60)
992     print "File Size (B): %d" % (file_size)
993     print "File Size (KB): %d" % (file_size_kb)
994     print "File Size (MB): %d" % (file_size_mb)
995     print "Average Speed (KB/s): %.2f" % (average_speed_kb)
996     print "Average Speed (MB/s): %.2f" % (average_speed_mb)
```

Figure B.10: Snippet 10 of client.py

```
998     # Append to csv file
999     csv_path = os.path.join(THESIS_DATA_PATH, 'data-download.csv')
1000    with open(csv_path, "a+b") as csv_file:
1001      w = csv.writer(csv_file)
1002      row = (file_size_mb, encrypted, CONST_ALPHA, CONST_K, CHUNK_SIZE,
1003             MINI_CHUNK_SIZE, number_of_nodes(), LOCALIZATION, protocol_total_time,
1004             find_total_time, download_total_time, find_breakdown, download_breakdown,
1005             misc_breakdown, average_speed_kb)
1006      w.writerow(row)
1007
1008  def download_chunk(server, port, master_file_id, file_id, file_size, current_size, encrypted, key=None):
1009    file_name = "chunk-%s" % (file_id)
1010    url_download = "http://%s:%s/file/%s" % (server, port, file_name)
1011
1012    # Chunk File
1013    chunk_file_path = os.path.join(my_node['file_path'], file_name)
1014
1015    # Buffer File
1016    buffer_file_path = os.path.join(my_node['file_path'], master_file_id)
1017    buffer_file_path = "%s.buffer" % (buffer_file_path)
1018
1019    r = requests.head(url_download)
1020    chunk_size = int(r.headers['content-length'])
1021
1022    mini_current_size = 0
1023    mini_chunk_size = min(chunk_size, MINI_CHUNK_SIZE)
1024
1025    if not os.path.isfile(buffer_file_path) or os.path.getsize(buffer_file_path) != file_size:
1026      upload_progress(file_id=master_file_id, reset=True)
```

Figure B.11: Snippet 11 of client.py

```
1028    if node_files.count(file_id=file_id) > 0:
1029        f = open(chunk_file_path, 'rb')
1030        partial_file_data = f.read()
1031        partial_file_data_size = len(partial_file_data)
1032        f.close()
1033
1034        # Unencrypt before adding to the buffer file
1035        if encrypted == ENCRYPTED_YES:
1036            unencrypted_chunk_file_path = os.path.join(my_node['file_path'], 'unencrypted.buffer')
1037            DecryptFile.decryptFile(key, chunk_file_path, unencrypted_chunk_file_path)
1038
1039            h = open(unencrypted_chunk_file_path, 'rb')
1040            partial_file_data = h.read()
1041            h.close()
1042
1043        # Buffer File
1044        try:
1045            g = open(buffer_file_path, 'r+b')
1046            g.seek(current_size)
1047        except IOError:
1048            g = open(buffer_file_path, 'wb')
1049        g.write(partial_file_data)
1050        g.close()
1051
1052        # Remove after adding its contents to the buffer video
1053        os.remove(unencrypted_chunk_file_path)
1054
1055        # Upload progress to own server
1056        current_size += partial_file_data_size
1057        file_progress = round(float(current_size)/file_size*100, 2)
1058        upload_progress(file_id=master_file_id, file_progress=file_progress, current_size=current_size,
1059                        file_size=file_size, current_chunk=file_name, has_key="Yes")
```

Figure B.12: Snippet 12 of client.py

```
1060    else:
1061      if encrypted == ENCRYPTED_YES:
1062        partial_file = urllib2.urlopen(url_download)
1063        partial_file_data = partial_file.read()
1064        partial_file_data_size = len(partial_file_data)
1065
1066        # Chunk File
1067        chunk_file_path = os.path.join(my_node['file_path'], file_name)
1068        f = open(chunk_file_path, 'wb')
1069        f.write(partial_file_data)
1070        f.close()
1071
1072        # Unencrypt before adding to the buffer file
1073        # key = file_keys.get_key(file_id=master_file_id)
1074
1075        unencrypted_chunk_file_path = os.path.join(my_node['file_path'], 'unencrypted.buffer')
1076        DecryptFile.decryptFile(key, chunk_file_path, unencrypted_chunk_file_path)
1077
1078        h = open(unencrypted_chunk_file_path, 'rb')
1079        partial_file_data = h.read()
1080        h.close()
1081
1082        # Buffer File
1083        try:
1084          g = open(buffer_file_path, 'r+b')
1085          g.seek(current_size)
1086        except IOError:
1087          g = open(buffer_file_path, 'wb')
1088        g.write(partial_file_data)
1089        g.close()
```

Figure B.13: Snippet 13 of client.py

```
1091            # Remove after adding its contents to the buffer video
1092            os.remove(unencrypted_chunk_file_path)
1093
1094            # Upload progress to own server
1095            current_size += partial_file_data_size
1096            file_progress = round(float(current_size)/file_size*100, 2)
1097            upload_progress(file_id=master_file_id, file_progress=file_progress, current_size=current_size,
1098                            file_size=file_size, current_chunk=file_name, has_key="Yes")
1099        else:
1100            while mini_current_size < chunk_size:
1101                req = urllib2.Request(url_download)
1102                req.headers['Range'] = "bytes=%s-%s" % (mini_current_size, mini_current_size+mini_chunk_size-1)
1103                partial_file = urllib2.urlopen(req)
1104                partial_file_data = partial_file.read()
1105                partial_file_data_size = len(partial_file_data)
1106
1107                # Chunk File
1108                try:
1109                    f = open(chunk_file_path, 'r+b')
1110                    f.seek(mini_current_size)
1111                except IOError:
1112                    f = open(chunk_file_path, 'wb')
1113                f.write(partial_file_data)
1114                f.close()
1115
1116                # Buffer File
1117                try:
1118                    g = open(buffer_file_path, 'r+b')
1119                    g.seek(current_size)
1120                except IOError:
1121                    g = open(buffer_file_path, 'wb')
1122                g.write(partial_file_data)
1123                g.close()
```

Figure B.14: Snippet 14 of client.py

```
1125                # Upload progress to own server
1126                current_size += partial_file_data_size
1127                mini_current_size += partial_file_data_size
1128                file_progress = round(float(current_size)/file_size*100, 2)
1129                upload_progress(file_id=master_file_id, file_progress=file_progress, current_size=current_size,
1130                                file_size=file_size, current_chunk=file_name, has_key="Yes")
1131
1132        # File size used for computing the progress
1133        return current_size
```

Figure B.15: Snippet 15 of client.py

B.2 Snippets of server.py

```
216 ##########################################################
217 # SECURITY
218 ##########################################################
219 def authenticate(func):
220   def validator(*args, **kwargs):
221     data = dict(request.forms)
222
223     if 'security_iv' in data and 'security_encryptedRandomKey' in data and \
224        'security_encryptedAESKey' in data and 'security_publicKey' in data and \
225        'security_signature' in data:
226
227       IV = data['security_iv']
228       encryptedRandomKey = data['security_encryptedRandomKey']
229       encryptedAESKey = data['security_encryptedAESKey']
230       rawPubKey = data['security_publicKey']
231       signature = data['security_signature']
232
233       prvKey = EncryptFunctions.loadKey(PRIVATE_KEY_PATH)
234
235       try:
236         AESKey = EncryptFunctions.decryptRSA(prvKey=prvKey, package=encryptedAESKey)
237         randomKey = EncryptFunctions.decryptAES(cipher=encryptedRandomKey, key=AESKey, iv=IV)
238         pubKey = EncryptFunctions.createRSAFromStr(rawPubKey)
239         verification = EncryptFunctions.verifyFunction(pubKey=pubKey, signature=signature, data=randomKey)
240       except:
241         print 'WARNING: YOU HAVE REJECTED A CONNECTION'
242         return 'REJECT'
243
244       if verification:
245         return func(*args, **kwargs)
246       else:
247         print 'WARNING: YOU HAVE REJECTED A CONNECTION'
248         return 'REJECT'
```

Figure B.16: Snippet 1 of server.py

```
250     else:
251       print 'WARNING: YOU HAVE REJECTED A CONNECTION'
252       return 'REJECT'
253
254   return validator
255
256 def deauthenticate(data):
257   del(data['security_iv'])
258   del(data['security_encryptedRandomKey'])
259   del(data['security_encryptedAESKey'])
260   del(data['security_publicKey'])
261   del(data['security_signature'])
```

Figure B.17: Snippet 2 of server.py

```
570  @get('/file-progress/<filename:path>')
571  def file_progress_get(filename):
572    if filename in file_progress:
573      return file_progress[filename]
574    else:
575      return { 'file_progress': 0.0,
576               'current_size': 0,
577               'file_size': 0,
578               'current_chunk': 'None',
579               'has_key': 'No' }
580
581  @post('/file-progress/<filename:path>')
582  def file_progress_post(filename):
583    progress = request.forms.get('file_progress')
584    current_size = request.forms.get('current_size')
585    file_size = request.forms.get('file_size')
586    current_chunk = request.forms.get('current_chunk')
587    has_key = request.forms.get('has_key')
588
589    if filename in file_progress:
590      if progress:
591        file_progress[filename]['file_progress'] = round(float(progress), 2)
592      if current_size:
593        file_progress[filename]['current_size'] = int(current_size)
594      if file_size:
595        file_progress[filename]['file_size'] = int(file_size)
596      if current_chunk:
597        file_progress[filename]['current_chunk'] = current_chunk
598      if has_key:
599        file_progress[filename]['has_key'] = has_key
```

Figure B.18: Snippet 3 of server.py

```
600    else:
601      file_progress[filename] = { 'file_progress': 0.0,
602                                  'current_size': 0,
603                                  'file_size': 0,
604                                  'current_chunk': 'None',
605                                  'has_key': 'No' }
606
607      if progress:
608        file_progress[filename]['file_progress'] = round(float(progress), 2)
609      if current_size:
610        file_progress[filename]['current_size'] = int(current_size)
611      if file_size:
612        file_progress[filename]['file_size'] = int(file_size)
613      if current_chunk:
614        file_progress[filename]['current_chunk'] = current_chunk
615      if has_key:
616        file_progress[filename]['has_key'] = has_key
617
618    return 'OK'
```

Figure B.19: Snippet 4 of server.py

B.3 Snippet of bootstrap.py

```
149  @post('/ping')
150  def ping():
151    print "PING!"
152    peer_id = request.forms.get('peer_id')
153    peer_server = request.forms.get('peer_server')
154    peer_port = request.forms.get('peer_port')
155    bucket_upsert(peer_id=peer_id, peer_server=peer_server, peer_port=peer_port)
156
157    number_of_nodes = bucket_col.count()
158    if number_of_nodes > 1:
159      for iteration in range(1, number_of_nodes+1):
160        x = random.randrange(1, number_of_nodes+1)
161        message = bucket_col.find_one({'db_id': x})
162        node_id = message['node_id']
163
164        if message != None and peer_id != node_id:
165          del message['last_updated']
166          del message['db_id']
167          del message['_id']
168          return message
169
170    return 'None'
```

Figure B.20: Snippet of bootstrap.py

B.4 Custom Libraries

B.4.1 EncryptFunctions.py

```
1  from Crypto.Hash import MD5
2  from Crypto.Hash import SHA256
3  from Crypto.Protocol.KDF import PBKDF2
4  from Crypto.Cipher import AES
5  from Crypto.PublicKey import RSA
6  from Crypto.Cipher import PKCS1_OAEP
7  from Crypto.Signature import PKCS1_v1_5
8  from Crypto.Util import number
9  import Crypto.Random.OSRNG.posix
10 import os
11 import base64
12 import GetSerial
13 from os.path import exists, join
14
15 BLOCK_SIZE = 16
16 KEY_SIZE = 32
17 PADDING = '/'
18
19 '''
20 hashToInt():
21 - this function converts the output of hashFunction() to int
22 - Usage: intConverted = hashToInt(outputOfHashFunction)
23 '''
24 def hashToInt(hashedString):
25     uInt = number.bytes_to_long(hashedString)
26     return uInt
27
28
29 '''
30 hashFcnSec():
31 - this function takes the SHA256 hash of the variable rawData and returns the hash
32 - Usage: hashedData = hashFcnSec(dataToHash)
33 '''
```

Figure B.21: Screenshot 1 of EncryptFunctions.py

```
34  def hashFcnSec(rawData):
35      h = SHA256.new()
36      h.update(rawData)
37      return h.hexdigest()
38
39  '''
40  hashFcnFast():
41  - this function takes the MD5 hash of the variable rawData and returns the hash
42  - Usage: hashedData = hashFcnFast(dataToHash)
43  '''
44  def hashFcnFast(rawData):
45      h = MD5.new()
46      h.update(rawData)
47      return h.hexdigest()
48
49  '''
50  generateIV():
51  - this function is used to generate the interrupt vector needed by AES encryption
52  - Usage: ivec = generateIV()
53  '''
54  def generateIV():
55      if os.name == "nt":
56          return os.urandom(BLOCK_SIZE)
57      else:
58          return Crypto.Random.OSRNG.posix.new().read(BLOCK_SIZE)
59
60  '''
61  generateAESKey():
62  - this function is used to generate a random AES key
63  - Usage: AESKey = generateAESKey()
64  '''
65  def generateAESKey(iterations=1024):
66      password = hashFcnSec(b'105@NDr0!dn37W0r&')
```

Figure B.22: Screenshot 2 of EncryptFunctions.py

```
67      key = ''
68      salt = os.urandom(32)
69
70      key = PBKDF2(password, salt, dkLen=32, count=iterations)
71      return key
72
73  '''
74  encryptAES():
75  - this function is used to perform AES encryption on the input data
76  - Usage: AESEncryptedData = encryptAES(dataToEncrypt, AESKey, ivec)
77  '''
78  def encryptAES(data, key, iv):
79
80      cipher = AES.new(key, AES.MODE_CBC, iv)
81
82      pad = lambda s: s + (BLOCK_SIZE - len(s) % BLOCK_SIZE) * PADDING
83
84      AESEncrypted = base64.b64encode(cipher.encrypt(pad(data)))
85
86      return AESEncrypted
87
88  '''
89  decryptAES():
90  - this function is used to decrypt AES ciphers
91  - Usage: decryptedAESData = decryptAES(obtainedCipher, AESKey, ivec)
92  '''
93  def decryptAES(cipher, key, iv):
94      decipher = AES.new(key, AES.MODE_CBC, iv)
95      DecodeAES = lambda c, e: c.decrypt(base64.b64decode(e)).rstrip(PADDING)
96      AESDecrypted = DecodeAES(decipher, cipher)
97      return AESDecrypted
98
99  '''
```

Figure B.23: Screenshot 3 of EncryptFunctions.py

```
100  exportPubKey():
101  - this function saves the generated public key into a file
102  - Usage: exportPubKey('filename.bla', RSAKey)
103  '''
104  def exportPubKey(filename, key):
105      try:
106          f = open(filename, 'w')
107      except IOError as e:
108          print e
109          raise
110      else:
111          f.write( key.publickey().exportKey("PEM") )
112          f.close()
113
114  '''
115  exportPrvKey():
116  - this function saves the generated private key into a file
117  - Usage: exportPrvKey('filename.bla', RSAKey)
118  '''
119  def exportPrvKey(filename, key):
120      try:
121          f = open(filename, 'w')
122      except IOError as e:
123          print e
124          raise
125      else:
126          f.write( key.exportKey("PEM") )
127          f.close()
128
129  '''
130  loadKey():
131  - this function loads exported keys
132  - Usage: loadKey('filename.bla')
```

Figure B.24: Screenshot 4 of EncryptFunctions.py

```
133 '''
134 def loadKey(filename):
135     try:
136         f = open(filename)
137     except IOError as e:
138         print e
139         raise
140     else:
141         key = Crypto.PublicKey.RSA.importKey(f.read())
142         f.close()
143     return key
144 
145 '''
146 loadRawKey():
147 - this function loads exported keys but does not convert into an RSA object
148 - Usage: loadRawKey('filename.bla')
149 '''
150 def loadRawKey(filename):
151     try:
152         f = open(filename)
153     except IOError as e:
154         print e
155         raise
156     else:
157         key = f.read()
158         f.close()
159     return key
160 
161 '''
162 createRSAFromStr():
163 - this function loads exported keys but does not convert into an RSA object
164 - Usage: createRSAFromStr('long rsa text')
165 '''
```

Figure B.25: Screenshot 5 of EncryptFunctions.py

```
166 def createRSAFromStr(str):
167     return Crypto.PublicKey.RSA.importKey(str)
168
169 '''
170 generateRSA():
171 - this function generates RSA keys and saves them in the present working directory
172 - Usage: generateRSA(fileToStorePubklicKeyIn, fileToStorePrivateKeyIn)
173 '''
174 def generateRSA(directory, pubFile, prvFile):
175     pubFile = join(directory, pubFile)
176     prvFile = join(directory, prvFile)
177     if not exists(pubFile) or not exists(prvFile):
178         RSAKey = RSA.generate(bits=2048, e=65537)
179         exportPubKey(pubFile, RSAKey)
180         exportPrvKey(prvFile, RSAKey)
181
182 '''
183 encryptRSA():
184 - this function encrypts data using RSA standards
185 - Usage: RSAEncryptedData = encryptRSA(loadedPublicKeyOfReceiver, encryptedSymmetricKeyToRSA)
186 '''
187 def encryptRSA(rcvPubKey, data):
188     # decodes data if it is base64 encoded
189     # data = base64.b64decode(data)
190     rcvPubKey = PKCS1_OAEP.new(rcvPubKey)
191     encrypted = rcvPubKey.encrypt(data)
192     return base64.b64encode(encrypted)
193
194 '''
195 decryptRSA():
196 - this function decrypts RSA encrypted data
197 - Usage: decryptedRSAData = decryptRSA(yourLoadedPrivateKey, RSAEncryptedData)
198 '''
```

Figure B.26: Screenshot 6 of EncryptFunctions.py

```
199 def decryptRSA(prvKey, package):
200
201     prvKey = PKCS1_OAEP.new(prvKey)
202     decrypted = prvKey.decrypt(base64.b64decode(package))
203     return decrypted
204
205 '''
206 signFunction():
207 - this function hashes input data and signs it using the owner's private key
208 - Usage: signedCertificate = signFunction(yourLoadedPrivateKey, encryptedSymmetricKeyToRSA)
209 '''
210 def signFunction(prvKey, data):
211     signer = PKCS1_v1_5.new(prvKey)
212     digest = SHA256.new()
213     # decodes data if it is base64 encoded
214     # digest.update(base64.b64decode(data))
215     sign = signer.sign(digest)
216     return base64.b64encode(sign)
217
218 '''
219 verifyFunction():
220 - this function hashes received input data from another user then signs it with
221 the owner of the signed certificate's public key, then compares it to the signed
222 certificate to verify the data being sent
223 - Usage: boolVar = verifyFunction(senderLoadedPublicKey, signedCertificate, encryptedSymmetricKeyToRSA)
224 '''
225 def verifyFunction(pubKey, signature, data):
226     signer = PKCS1_v1_5.new(pubKey)
227     digest = SHA256.new()
228     # Assumes the data is base64 encoded to begin with
229     # digest.update(base64.b64decode(data))
230     if signer.verify(digest, base64.b64decode(signature)):
231         return True
```

Figure B.27: Screenshot 7 of EncryptFunctions.py

```
232         return False
233
234     '''
235     nodeID():
236     - this function generates a tuple consisting of the node ID in string form and node ID in integer form
237     - Usage: nodeStr, nodeInt = nodeID()
238     '''
239     def nodeID():
240         myserial = GetSerial.getSerialDev()
241
242         if myserial == '0000000000000000':
243             myserial = GetSerial.getSerialVM()
244
245         nodeStr = hashFcnSec(myserial)
246         nodeInt = hashToInt(nodeStr)
247         return nodeStr, nodeInt
248
249
250     '''
251     serialID():
252     - this function generates the serial ID for your node
253     - Usage: myserial = serialID()
254     '''
255     def serialID():
256         myserial = GetSerial.getSerialDev()
257
258         if myserial == '0000000000000000':
259             myserial = GetSerial.getSerialVM()
260
261         return myserial
262
263     '''
264     nodeIDVM():
```

Figure B.28: Screenshot 8 of EncryptFunctions.py

```
265  - this function generates a tuple consisting of the PSEUDO node ID in string form and node ID in integer form
266  - Usage: nodeStr, nodeInt = nodeIDVM()
267  '''
268  '''
269  def nodeIDVM():
270      mvserial = GetSerial.getSerialVM()
271      nodeStr = hashFcnSec(mvserial)
272      nodeInt = hashToInt(nodeStr)
273      return nodeStr, nodeInt
274  '''
275
276  '''
277  fileID():
278  - this function returns the file ID of a given input file in string form and integer form
279  - Usage: fileIDStr, fileIDInt = fileID(filePath)
280  '''
281  def fileID(filePath):
282      h = MD5.new()
283      chunk_size = 2<<25
284
285      with open(filePath, 'rb') as fileToRead:
286          while True:
287              chunk = fileToRead.read(chunk_size)
288              if len(chunk) == 0:
289                  break
290              h.update(chunk)
291
292      fileIDStr = hashFcnFast(h.hexdigest())
293      fileIDInt = hashToInt(fileIDStr)
294      return fileIDStr, fileIDInt
295
296  if __name__ == "__main__":
297      pass
```

Figure B.29: Screenshot 9 of EncryptFunctions.py

B.4.2 GetSerial.py

```
1  from os.path import exists
2  '''
3  getSerialDev():
4  - gets the unique serial number of the Pi as a string
5  - usage: mvserial = getSerialDev()
6  '''
7  def getSerialDev():
8    serial = "0000000000000000"
9    try:
10     f = open('/proc/cpuinfo','r')
11     for line in f:
12       if line[0:6]=='Serial':
13         serial = line[10:26]
14     f.close()
15   except:
16     serial = "ERRORXXXXXXXXXX"
17
18   return serial
19
20 '''
21 getSerialVM():
22 - generates a random pseudo serial number for the VM's as a string
23 - usage: mvserial = getSerialVM()
24 '''
25 def getSerialVM():
26   fname = 'pseudo_id.idf'
27   if not exists(fname):
28     #print 'not existent yet'
29     import string
30     import random
31     serial = ''.join(random.choice(string.ascii_uppercase + string.digits) for x in range(16))
32     #print serial
33     try:
```

Figure B.30: Screenshot 1 of GetSerial.py

```
34          f = open(fname, 'wb')
35        except IOError as e:
36          print e
37          raise
38        else:
39          f.write(serial)
40          f.close()
41      else:
42        #print 'file exists already!'
43        try:
44          f = open(fname, 'rb')
45        except IOError as e:
46          print e
47          raise
48        else:
49          serial = f.read()
50          f.close()
51      #print serial
52      return serial
53
54  if __name__ == "__main__":
55      myserial = getSerial()
56      print myserial
```

Figure B.31: Screenshot 2 of GetSerial.py

B.4.3 EncryptFile.py

```
12  def encryptFile(key, inFilename, csize=64*1024*1024):
13      outFilename = "%s.enc" % (inFilename)
14
15      iv = EncryptFunctions.generateIV();
16      encryptor = AES.new(key, AES.MODE_CBC, iv)
17      filesize = os.path.getsize(inFilename)
18
19      with open(inFilename, 'rb') as infile:
20          with open(outFilename, 'wb') as outfile:
21              outfile.write(struct.pack('<Q', filesize))
22              outfile.write(iv)
23
24              while True:
25                  chunk = infile.read(csize)
26                  if len(chunk) == 0:
27                      break
28                  elif len(chunk) % 16 != 0:
29                      chunk += ' ' * (16 - len(chunk) % 16)
30
31                  outfile.write(encryptor.encrypt(chunk))
32
33      # Replace original file with the encrypted version
34      os.remove(inFilename)
35      os.rename(outFilename, inFilename)
```

Figure B.32: Screenshot of EncryptFile.py

B.4.4 DecryptFile.py

```
11  def decryptFile(key, inFilename, outFilename=None, csize=64*1024*1024):
12      outFilenameIsNone = False
13      if not outFilename:
14          outFilename = "%s.unc" % (inFilename)
15          outFilenameIsNone = True
16
17      with open(inFilename, 'rb') as infile:
18          origsize = struct.unpack('<Q', infile.read(struct.calcsize('Q')))[0]
19          iv = infile.read(16)
20          decryptor = AES.new(key, AES.MODE_CBC, iv)
21
22          with open(outFilename, 'wb') as outfile:
23              while True:
24                  chunk = infile.read(csize)
25                  if len(chunk) == 0:
26                      break
27                  outfile.write(decryptor.decrypt(chunk))
28
29              outfile.truncate(origsize)
30
31      # Replace original file with the unencrypted version
32      if outFilenameIsNone:
33          os.remove(inFilename)
34          os.rename(outFilename, inFilename)
```

Figure B.33: Screenshot of DecryptFile.py